UMONS
Université de Mons

Faculté Polytechnique

Creating an automatic annotation pipeline for human body members

End of studies work presented with a view to obtaining the degree of Master of Civil Electrical Engineering, with a specialized focus on Artificial Intelligence and Smart Communication

Benoît VIDOTTO

POLYTECH MONS

Under the direction of:
Mr. Professor Bernard GOSSELIN
Mr. Kevin EL HADDAD
Madame Virginie VANDENBULCKE
Academic Year 2021-2022

ACADÉMIE UNIVERSITAIRE WALLONIE-BRUXELLES

Résumé

This project is in the context of creating a human body member detection system. Such a system would enrich several applications in different fields such as search engines, human-agent interaction systems, etc. The goal is to create an object detection pipeline without using manually annotated databases (automatic annotations). In order to create machine learning models for object detection, a large amount of data containing annotated images is required. Creating such databases, i.e. manually collecting and annotating images, is time-consuming and tedious. With the growing popularity of object detection, many scientists are looking for ways to facilitate the annotation of databases. This work explores a supervised training method without manual annotation for human body limb detection.

After a first state-of-the-art chapter presenting the existing solutions in this field, different databases were explored as well as the annotation formats associated with them. If these databases were to be incomplete or required modifications, this second chapter is also devoted to the study of three annotation tools which allowed to choose the best of them. Finally, this chapter ends with the presentation of a method of collecting images from the internet called "data scraping".

The presentation and study of the machine learning object detection models NanoDet and YOLO as well as the annotation method using MediaPipe allows to determine the best detector of human body parts. The presentation of the metrics for the object detection models explains the use of the mean Average Precision score (mAP) as a function of the Intersection over Union (IoU) threshold. After studying the performances of the three object detection systems all three applied on the same Pascal-Part database, the YOLO system is superior to the two other systems NanoDet and MediaPipe.

Using the results of the comparison of object detection systems, an automatic annotation pipeline was created using data scraping, YOLO and MediaPipe. The database created using data scraping was built on the DuckDuckGo search engine using the keywords "sitcom scene". After being automatically annotated using MediaPipe, it is used to train the object detection model YOLO. The trained model is validated on the Pascal-Part database. Better detection results are achieved by replacing the database from data scraping with a larger database from the video hosting website YouTube. Finally, training the system without distinguishing between left and right limbs of the human body further improves the results.

Acknowledgments

To Professor Bernard Gosselin for giving me the opportunity to complete this final year thesis at the ISIA Lab.

To Mr. Kevin El Haddad and Mrs. Virginie Vandenbulcke for their availability and their valuable advice during the completion of this work.

To my loved ones for the support and guidance provided during the completion of this work.

Glossaire

artificial neural network Translated from *Artificial Neural Network (ANN)* and inspired by a biological neural network, it is a set of computational nodes (neurons) arranged in layers and connected to each other in order to learn the characteristics of a database.

Average Precision Translated from English as average precision, Average Precision is a metric used to evaluate the detection of a category of objects within the framework of an object detection model. This metric is determined by the area underlying the precision-recall curve.

bounding rectangle Used in English by the terms *bounding box*, it is the rectangle which delimits an object in an image allowing to define the location of this one in the image.

COCO Acronym for Common Objects in COntext, it is a database for object detection.

convolutional neural network Translated from *Convolutional Neural Network (CNN)*, this is a artificial neural network comprising filters called kernels allowing to extract the relevant features of an image by using convolution operations.

cost function Function indicating the progress and maturity of training a machine learning model.

data scraping Automatic data collection method found on the internet.

deep learning Machine learning method in which unstructured data is provided to the model.

early stopping Machine learning method in which the model stops learning if no improvement is observed.

epoch In a machine learning pipeline, the number of epochs determines the number of times the model runs through the entire training database in order for it to learn the characteristics of the data.

fine tuning In a machine learning model, fine tuning allows the weights of the model to evolve only on the last layers of the model and to freeze the weights of the other layers. This is a refinement of the model.

inference Running a machine learning model once it has been trained.

Intersection over Union Intersection over Union (IoU) is a term used to describe the extent to which two rectangles overlap. Given the two rectangles, the more the second rectangle reproduces the characteristics (in shape, size and position) of the first, the greater the IoU will be. If the two rectangles have the same shape, size and position, the IoU will be 1, otherwise 0.

mean Average Precision Average of the Average Precision of each object category in a database.

overfitting A phenomenon in machine learning in which the model is not able to extrapolate the learned information to databases other than the one used during training.

Pascal-Part Database for object detection in which objects are annotated using segmentation masks.

pipeline In machine learning, a pipeline consists of the steps needed to train and validate a model.

segmentation mask A form of image annotation in which objects are annotated pixel by pixel.

supervised learning Method of training a machine learning model based on annotated data.

Support Vector Machine A family of machine learning algorithms that use support vectors to differentiate between two categories of data by calculating the distance between the two groups.

trust The confidence score reflects the probability that the bounding rectangle contains an object and the positional precision of this rectangle.

unsupervised learning A method of training a machine learning model in which the model finds the different characteristics of the data by itself.

YOLO Acronym for You Only Look Once, it is a one-step object detection algorithm.

Acronymes

ANN Artificial Neural Network.
AP Average Precision.

CNN Convolutional Neural Network.

IoU Intersection over Union.

mAP mean Average Precision.
MNIST Modified National Institute of Standards and Technology database.

Pascal Pattern Analysis, Statistical Modeling and Computational Learning.

SVM Support Vector Machine.

VOC Visual Object Classes.

Table des matières

Table des figures

Introduction

In recent years, object detection, a branch of computer vision, has become increasingly popular. Object detection consists of localizing and classifying objects present in one or more images, usually using a machine learning model (*machine learning*). In order to teach the model to detect the desired objects, it is trained using images annotated with the chosen objects. To have an efficient object detection model, many annotated images are required for training. Manually annotating such a large number of images for this purpose is a long and tedious job that researchers try to avoid using automatic annotation models.

Human detection is a popular category of object detection due to the many possible applications. The applications studied are pedestrian detection for the autonomous driving of tomorrow's cars, the improvement of video surveillance data or the analysis of behaviors in various environments such as in shopping centers for sales purposes or in industrial environments for security purposes. Additional information to these applications can be provided through pose estimation or the detection of human body parts.

This work aims to provide a comparison of machine learning models for object detection (in particular human body parts) and to create a pipeline for automatic detection and annotation of human body parts in order to create a database automatically annotated by this pipeline. The first chapter studies the existing solutions for object detection and automatic annotation of human body parts in the literature, the second compares the different databases for detection of human body parts and their annotation formats, the third compares the object detection algorithms and the last chapter presents the creation of the pipeline for automatic annotation as well as its results against a real database and object detection model.

The contributions made by this work are :
- Comparison of database annotation tools ;
- the use and comparison of object detection systems ;
- the implementation of an image collection tool on the internet ;
- the creation of a pipeline for automatic object detection and annotation using machine learning.

Chapitre 1

State of the art

1.1 Object Detection

1.1.1 Neural Networks

The machine learning models (*machine learning* models) used for this work are based on the concept of artificial neural network (*Artificial Neural Network (ANN)*). Inspired by a biological neural network, a artificial neural network is a set of computational nodes (neurons) arranged in layers and connected to each other in order to learn the characteristics of a database. Each computational node is associated with a value called a weight that evolves according to the result of the network in order to optimize it. This evolution of the weights is called training. In a supervised learning, the evolution of the training of these weights is evaluated using a cost function that evaluates the accuracy of the result of the neural network. In order to learn, the artificial neural networks need a data set on which to train (training data set) and another on which to evaluate the learning (validation data set). If the results of the neural network are good on the training dataset but bad on the evaluation (or validation) dataset or on another dataset, then we speak of overfitting : the learning fits too much to the training database and cannot adapt to new data [1][2].

convolutional neural networks (*Convolutional Neural Network (CNN)*) are similar to artificial neural networks but are used more for image analysis. convolutional neural networks have the particularity of including filters called kernels (*kernels*) allowing to extract the relevant features of an image by using convolution operations. These filters thus allow machine learning models to more easily detect the elements of an image that will lead to their classification [1][2].

Neural networks can be trained using three different methods : supervised, semi-supervised and unsupervised. supervised learning is learning using annotated input data that are the expected results of the model. The goal of supervised learning will therefore be to reduce the classification error of the neural network by establishing the weights of each neuron (cost function). unsupervised learning does not provide any annotation to artificial intelligence. The objective of the system is then to associate elements among the data with the same characteristics. The cost function of the system describes the quality of the associations created by artificial intelligence [2]. Finally, semi-supervised learning is a hybrid form of the two

previous methods : the system only receives part of the annotations, it is up to the system to determine the annotations of the unannotated data [1].

When the object detection machine learning model has been trained and validated using a database and has learned the features of the database, inference is the detection of objects in an image, whether or not from the training and/or validation database.

1.1.2 History and applications

Object detection is a branch of deep learning aimed at identifying and classifying objects present in an image. More precisely, object detection originates from computer vision and is divided into different tasks, including object detection, such as classification, semantic segmentation and instance segmentation (see figure 1.1)[3]. Interest in object detection emerged in the 1990s and is increasingly popular with the rise of related applications such as pedestrian and car detection for autonomous driving, face detection or text detection [4]. Object detection allows to perform different tasks as the COCO[5] database does : human pose estimation which aims to match all human pixels of a color image with the 3D surface of the human body [6] ; panoptic segmentation which combines semantic and instance segmentations [7] ; keypoint detection, such as joints of the human body.

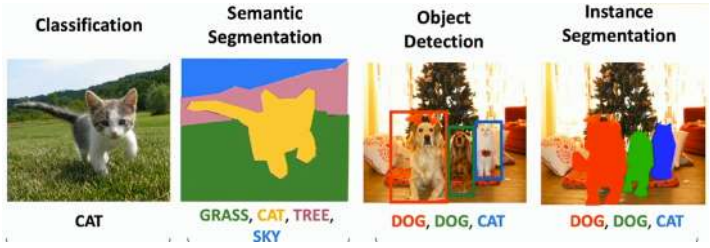


FIGURE 1.1 – Different components of computer vision (classification, semantic segmentation, object detection, instance segmentation)[8]

Image detection using neural networks begins with the detection of handwritten digits in 1989 by Yann LeCun et al. using the backpropagation algorithm (*backpropagation algorithm*), an algorithm that updates the weights of a neural network based on the classification errors made by the [9] network. The neural network created has one input for each pixel of the image and 10 outputs, one output per decimal digit. This neural network was trained using the MNIST database.

Viola et al. developed the first real-time object detector to detect human faces in 2001. The detector works using a sliding window and checks whether a face appears in each window. Since using sliding windows is not viable for real-time detection (this technique is computationally intensive), the authors used three techniques to improve the process, including avoiding having the sliding windows target the background of an image [10].

RCNN (*Regions with Convolutional Neural Network (CNN) features*) is the first object detector using convolutional neural networks. The method proposed by [11] has improved detection by 30

Single-stage object detectors appeared in 2015 with the advent of YOLO. The two stages required for the operation of two-stage detectors are combined into a single convolutional neural network, hence the name YOLO : You Only Look Once. This innovation does not improve the accuracy and classification quality of two-stage object detectors (especially on small objects) but greatly optimizes the runtime allowing the detection of a large number of objects in real time [12]. Other famous single-stage object detectors are the four subsequent versions of YOLO which improve the detection quality, as well as SSD (*Single Shot MultiBox Detector*) and RetinaNet [4].

A large number of applications make use of object detectors such as pedestrian, face, text or road sign detection. Pedestrian detection can be assimilated to human detection and faces certain challenges [4] : the detection of "small" pedestrians (few pixels define the pedestrian), the detection of absolute negatives (the model is informed that some objects are not to be confused with human beings) and the detection of groups of pedestrians. In order to improve the detection of small pedestrians, researchers use two-stage object detectors because they are more efficient at detecting small objects than single-stage detectors and other techniques such as feature fusion, and other tricks on image resolution. Absolute negative detection is improved using decision trees (*decision trees*) and pedestrian groups are detected using a new cost function. Road sign detection under varying illumination has been improved using generative adversarial networks (*GANs*). The authors of [13] improve face group detection under occlusions by classifying face parts and their spatial position.

1.2 Human and Body Limb Detection

DT Nguyen et al. did a study on human detection as object detection [14]. Human detection is more complicated than inert objects because of the large number of possible poses, clothing and colors, shapes, even illumination. Humans are actually able to detect humans using subtle cues that can be complicated for a machine learning model to learn.

The steps for detecting humans are as follows (see figure 1.2) :
- extraction of candidate regions that probably contain humans,
- the description of the extracted regions,
- classification/verification of regions as "human" (containing a human) or "non-human",
- and the post-processing which allows to specify the detection.

These steps are therefore similar to the detection of a classical object. Human detection can also be performed using background extraction methods. Human detections are done using feature extraction such as the shape of the human and its edges at pixel-level or regions, motion (in the case of videos), appearance, etc. These features can be found using concepts such as Histogram of Oriented Gradients (HOG), Principal Component Analysis (PCA), Histogram Of Flows (HOF) and other image transformations. Human descriptions are constructed by combining features of local regions based on grids covering the human or based on salient points. Human classification can be generative or discriminative. The generative approach to human detection is achieved by building a model of the object under study, whether this model is of shape, structure or appearance. The discriminative method consists of learning to detect humans, using support vector machines (*Support Vector Machine (SVM)*) or artificial neural networks. Discriminative approaches using neural networks have been particularly effective

against older methods for pedestrian detection, although the small size of some objects can be problematic.

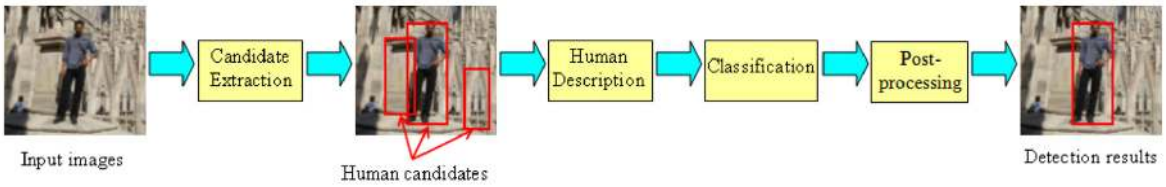


FIGURE 1.2 – Steps in detecting a human [14]

One of the challenges of human detection is occlusion, that is, when a human is partly hidden and therefore partly invisible. Where a human will extrapolate information using missing data, the computer will not detect a human being who is partly invisible. Occlusion can be of different kinds : a human can be partly hidden by another human, by himself or by an object. The shooting also plays a role depending on the framing and the viewing angle. Several techniques for detecting humans under occlusion exist and fall into two categories : methods based on detection and methods based on inference. Methods based on detection only use the object and its different parts. As an example, the authors of [15] propose an occlusion management based on the recognition of human body parts and their position relative to each other. Methods based on inference are based on the mutual relationship between a human being and other objects. This approach is often carried out in cases of inter-object occlusion, i.e. between humans. The performance of occlusion techniques strongly depends on the quality of the human body part classifiers.

Human detection in sports activities is more challenging for models due to the variability of different poses and occlusion between humans. To control this problem, said activities are classified and annotated in the associated images. Human body limb detection must be done with dense and complicated models capable of supporting the complexity of the task [16].

Following the study of the different existing models and techniques for human detection, Nguyen et al. [14] propose the use of some models exclusively for certain applications, for example, depending on the number of different poses, the desired response time or for the detection of body members. For applications that involve a large number of different poses, some models that are proposed use "poselet" techniques and support vector machines (*Support Vector Machine (SVM)*). The models suggested for pedestrian detection are simpler and are created using histograms of oriented gradients (*Histogram of Oriented Gradients (HOG)*, support vector machines or pseudo-Haar features. In order to detect body parts, the authors of [17] used object detection and articulated pose estimation as well as sampled descriptors on the Adaboost discriminative classification training method. The authors of [16] present a model that uses the orientation of body parts instead of the joints between the parts. This use of the orientation of the parts is mixed with models for each part of the body. The authors notably use a database extracted from a television series and manage to improve the previous scores by 50

1.2.1 Human Pose Estimation

Human pose estimation involves the detection of human bodies and their limbs and their position in space. Human pose estimation is useful for various applications such as motion capture in the film and video game industry, in action detection and prediction, or even in healthcare. Figure 1.3 presents the different ways to represent human poses. The kinematic model is visible in figure 1.3.a. This model consists of a set of points of interest, often placed at the joints of the human skeleton, and connected to each other by line segments, similar to the bones of the human skeleton [18]. Widely used for its simplicity, this intuitive model of the human body does not present texture and shape information. The planar model in figure 1.3.b is used to represent the shape and appearance of the human body by representing rectangles or polygons that are approximations of the limbs. The volumetric model in figure 1.3.c is mainly used in three-dimensional contexts such as animated films. This model offers a very accurate representation of the human body and allows to analyze how a body deforms when changing pose [19].

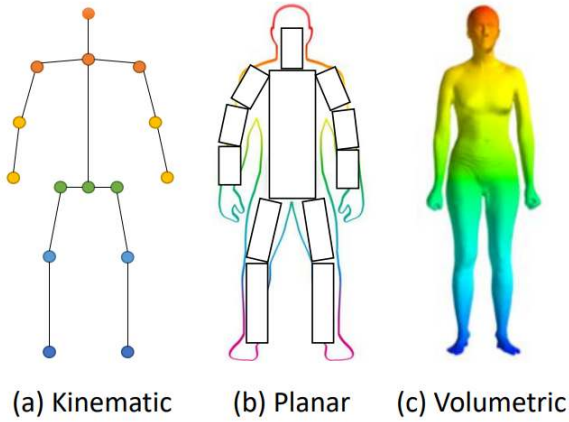


FIGURE 1.3 – Human pose representation models [19]

Two methods exist to estimate the pose of a single human in a two-dimensional context such as images : regression and heatmap methods. Regression methods use deep neural networks such as AlexNet, GoogLeNet and ResNet using convolutional neural network. Other regression techniques use multi-task learning in which the tasks of joint coordinate prediction and body part detection are performed using sliding windows. Heatmap methods aim to train a body part detector to predict the positions of body joints. This is done by predicting a heatmap for each keypoint to be detected and the training is done by minimizing the errors between the real and annotated heatmaps. The authors of [20] combine a body part detector based on a CNN and other models to detect skeleton joints. In addition, several works present human pose estimation using generative adversarial networks (GANs) in which the generator estimates the location of points and the discriminator compares the actual and predicted heatmaps. Other works attempt to create more information about the body structure by studying the complex relationships between body parts [19]. BlazePose estimates human poses using both heatmap and regression methods. Their model can detect 33 key body points in real time and on mobile devices [21].

Estimating the pose of multiple people is more complicated because it is necessary to determine the number of people present in an image as well as their position. Two methods exist for estimating the pose of multiple people : the top-down method and the bottom-up method. The top-down method uses an object detector applied to humans to detect their positions. The pose estimation will then be performed in the parts containing one and only one human being. The object detector used is often a machine learning model. Occlusion management is of particular importance when estimating the pose of multiple people because some people may be partially hidden by others. The authors of Personlab propose occlusion management using the Faster RCNN model. The bottom-up method is done in two steps : the detection of key points of the body and the assembly of these key points to individuals. This method is used by OpenPose [22] and Deepcut [23] which uses Faster RCNN to solve this challenge.

1.2.2 Human body part detection

Detecting human body parts requires complex and tedious annotation work. While this work can be completed by Amazon Mechanical Turk (AMT) volunteers, some work attempts to create these annotations automatically.

The authors of [24] use a semi-supervised method to generate synthetic human body segmentation data using keypoint annotations of human bodies, easily obtained using skeletons shown in figure 1.3.a already annotated. Their key idea is to transfer the annotations of body parts from one person to another with a similar pose. The steps are therefore to find people with similar poses, align the two poses and apply a refinement neural network (*fine tuning*) to estimate the body part segmentation. The refinement neural network is a machine learning model called "U-Net". In order to train this model, the Pascal-Person (dataset for body part segmentation [25]) and MPII (dataset containing multiple activities performed by humans [26]) databases were used.

[27] explain that in order to detect body parts at pixel level, previous studies have used synthetic data in order to avoid annotation of these data. The results of such experiments are worse than with real data. In order to improve these annotations, the authors try to combine synthetic data and existing real data using pose skeletons. The synthetic data were created using 20 3D human models placed in a virtual room and animated according to different actions. The real data are taken from the Pascal-Person and COCO-DensePose databases (dataset for body part segmentation [5]). In order to train their model, the ResNet machine learning model is used, improving it with elements of the OpenPose architecture. In addition, the system has improved the detection of key points on the human body. Thanks to this, the performances obtained are better than on the compared systems.

The authors of [28] attempt to improve body limb annotations by correcting them with a cyclic learning model. Cycles allow annotations to improve themselves at each period. The model is composed of two steps : model aggregation and annotation refinement. In order to achieve the presented objectives, the "Augmented-CE2P" model is created on the Pascal-Person and Look-Into-Person databases (body limb segmentation and pose estimation dataset [29]). The model data is intentionally noisy in order to expose the correction applied by the model. In addition, the model differentiates body limbs with the clothes worn by the human being.

Similar to [28], the authors of [30] correct the annotations of the body parts. The approach is different : the model assembles the information from three different processes. The first process is a direct prediction of each part of a human body using the information from the image. The second and third processes make use of the hierarchy in the arrangement of the human body parts : the foot is a subpart of the leg which is a subpart of the human body. Thanks to this hierarchy, the last processes make use of a bottom-up inference (the foot helps to detect the leg) and a top-down inference (the body helps to detect the leg). Based on ResNet, the network is trained using the Pascal-Person and Look-Into-Person databases. The methods used by this study outperform other existing models, including the model presented by [28].

Chapitre 2

Data Acquisition

In order to best meet the objectives of this work which deals with the recognition of human body parts via automatic model learning, it is necessary to obtain one or more databases containing annotations of human body parts. The first section of this chapter *Data acquisition* explores the different existing databases and the associated annotation formats are presented in the second section. If no database exists or if it is necessary to improve the existing annotations, annotation tools can be compared in the third section to determine the most suitable tool for this task. The last section shows that it is possible to create databases by automatically collecting images from the internet and annotating them with the previously mentioned tools.

2.1 Existing database

To be able to effectively identify body parts, it is necessary to find databases of human body parts in order to train machine learning models from this data. To do this, it is necessary to obtain as much data as possible with as many annotations as possible. These annotations must be in formats that are easily manipulated for modification and conversion into other formats. The available images should ideally be in color in order to provide more information to artificial intelligences and be as varied as possible in order to avoid the phenomenon of overfitting. Varied images are images for which the background varies between images and in which the subjects of the images occupy different and varied poses depending on the lighting, the shooting, the clothing, the colors, etc. Ideally, the number of annotations per image should also vary and the images should come from everyday life in order to avoid overly specific situations. The databases searched are therefore large and contain many images associated with many annotations whose categories are equally distributed among themselves, in colors and varied in all areas, i.e. in colors, backgrounds, poses, illuminations, scenes, etc.

The authors of [31] propose a database representing different seated people, seen from above in a disaster and according to the distance (see figure 2.1). These images are annotated either according to four categories (arms, legs, torso and head) or according to 14 categories (upper and lower left and right arms, upper and lower left and right legs, left and right hands and feet, torso and head). These databases are composed of only about 1000 images and the photographs often represent the same people with little variation in the backgrounds.

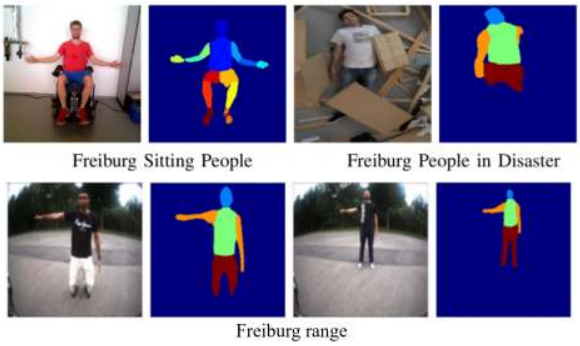


FIGURE 2.1 – Different components of the Freiburg database and their segmentation masks [31]

The authors of [32] present DID-Net, which is a model created to improve object detection performance for multi-level objects, i.e. objects that are part of a whole, such as the wheels of a car. To illustrate their point, the authors created a database called *Human-parts* containing 14,962 images, which were annotated with people and their hands and heads, but without including any other annotations for the body parts.

The authors of the paper [26] solicited Amazon Mechanical Turk volunteers to annotate 25,000 images containing 40,000 people in a database called *MPII Human Pose Dataset* (figure 2.2). The images were systematically collected using an established taxonomy of everyday human activities using the video hosting website YouTube. Image collection was done using the activities describing the collected image : images were captured from YouTube videos, up to 10 videos were selected per activity, with a five-second interval between each image capture. Subsequently, Amazon Mechanical Turk volunteers annotated these images with the activity depicted in the image and the human body limbs, as well as the human body skeleton using body joints and head and torso orientations.

The *Pascal Visual Object Classes* (VOC) [33] project was created in 2005 to establish a database for computer vision. In order to create this, this project consisted of challenges to be taken up by anyone wishing to participate in order to improve these annotations. The Pascal project thus provides standardized sets of image data for object class recognition, it also provides a common set of tools to access the data sets and annotations allowing the evaluation and comparison of different methods. From 2005 to 2012, annual challenges have increased the number of annotation categories as well as the number of images in the database. The authors of [34] use the Pascal VOC database to detect animal body parts in order to obtain more context regarding animal detection. This annotation is done using the model created by the authors. The researchers of [25] make use of the discoveries made by [34] : three inference processes are used to detect the parts of an object. The authors thus manage to segment the 20 categories present in the database. The database thus created *Pascal-Part* therefore has 20 super-categories of objects, each super-category comprising different sub-categories. For example, *Pascal-Part* illustrated in figure 2.3 has the annotations of an airplane but also of its different wings ; a car with its doors and wheels ; or a bird with its legs, head and wings ; etc. The annotations of the human body parts are the head (hair, ears, eyebrows, eyes, nose and mouth), the torso, hands, arms (upper and forearms), feet and legs (upper and lower).

FIGURE 2.2 – Samples from the *MPII Human Pose Dataset* database by activity [26]

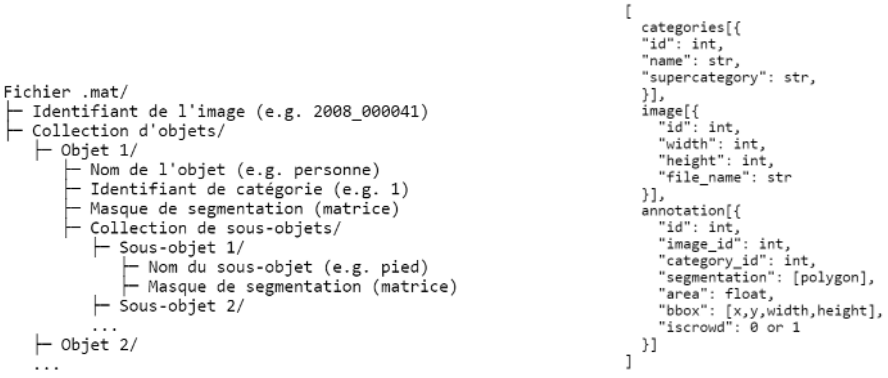
Each category of the human body parts are differentiated according to the left or right side when possible (left hand, right upper leg, ...). These images are annotated using segmentation masks, defined by a set of pixels in which the object concerned is located. The outcome of the Pascal VOC project being to create the best possible database, the images and their contents are varied and in sufficient number : the *Pascal-Part* database has 10103 images.

FIGURE 2.3 – Samples from the *pascal-part* database

Similar to the principles of the Pascal VOC challenge, the COCO database has organized challenges every year since 2015 in object detection, key points, subtitling or image description, scene detection or even human poses. Annotated by Amazon Mechanical Turk volunteers [35], the COCO database features a database of over 330,000 images, 1.5 million object instances, 80 categories, and object segmentations using masks. COCO also provides tools to manage the database format created specifically for it.

2.2 Data format

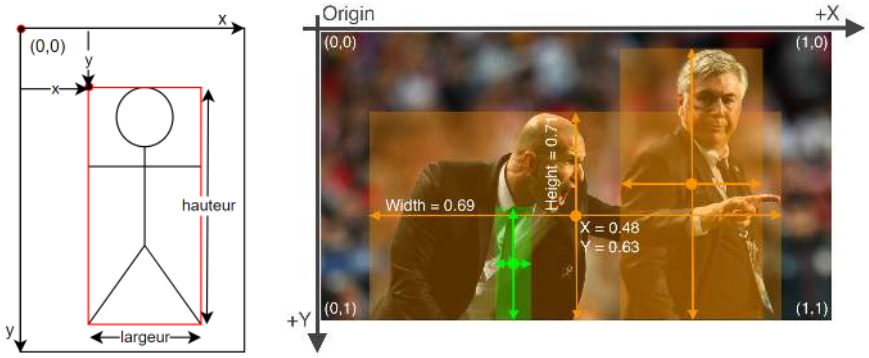
The Pascal-Part database is provided with its own notation system : each image is associated with a .mat file. This type of file is a binary data container format used by the MATLAB program. It allows a large amount of information to be saved using little storage. The architecture used by Pascal-Part is shown in figure 2.4.a. This structure contains the image identifier, the name of the object that is annotated on the image, as well as the category identifier, the segmentation mask and the sub-objects, and their own name and segmentation mask of these sub-objects. These types of annotations therefore have categories such as "airplane" or "human" and sub-categories such as "wing" or "foot". The segmentation masks are matrices composed of 1 and 0 whose dimensions are the same as the dimensions of the annotated image. In this matrix, the pixels delimiting the object take the value of 1, the others take a zero value.



has. Pascal-Part annotation format (.mat) b. COCO annotation format (.json)

FIGURE 2.4 – Annotation file structure

The Pascal-Part annotation system is not widely used by object detection models, so it is necessary to transform the data into a more popular format. The COCO database provides annotation tools and format that are very popular in the object detection algorithm community. A large majority of models support this annotation system due to its ease of use and completeness of features. COCO encapsulates the annotation data in a .json file. This type of file allows to save data structures such as variables, lists or dictionaries in a file that is easily accessible by computer programs. The structure of a .json annotation file is shown in figure 2.4.b. In addition to information data (such as year, author or description) or license, the annotation file presents three parts of classifications : the categories, which contain the name, identifier and derived subcategories ; the images which contain the dimensions of the image, file name and identifier ; and the annotations themselves, which contain the identifiers of annotations, category and image, the segmentation mask, the area that the object takes in the image, the bounding rectangle (*bounding box*) the object (bbox) and a parameter (iscrowd) which determines if this object is a set of several equal objects. The data in the annotation files of COCO are therefore numerous, easily accessible for anyone who comes to manipulate such data. The bounding rectangles of the object in the annotations are represented using a point defined at the top left of the rectangle and the height and width of the same rectangle (see figure 2.5.a). The convention is that the origin of a coordinate system in an image is at the top left corner of the image.



has. COCO b. YOLO [36]

FIGURE 2.5 – Delimitation of an object using a rectangle according to COCO (a) and YOLO (b)

YOLO is another artificial intelligence explored in more detail in the next chapter. The annotation format associated with this object detection model is different from the one proposed by COCO : that of YOLO is simpler but less complete. Each annotation text file is associated with an image from the database in which each line defines the bounding rectangle of an object : each line begins with the category number of the object followed by the four values defining the location of the object (eg : 14 0.89 0.4 0.1 0.2). The bounding rectangle is defined differently than in COCO : objects in an image are identified by the point located at the center of the object relative to the origin of the image and by the height and width (see figure 2.5.b). These data are then normalized by the height and width of the image. YOLO finally has a file that allows the algorithm to find the position of images and annotation files in file folders as well as to associate category identifiers with their real names allowing the user to identify these different categories.

2.3 Annotation Tools

This work aims to automatically annotate a database. It is very likely that the generated annotations are not perfect and that manual corrections are necessary. This section allows you to browse the most promising manual annotation tools. Several criteria allow you to choose among different tools available online :

— these must be open-source,
— ideally developed in the Python programming language,
— they allow the annotation of segmentation masks and not only of bounding rectangles,
— must be easy to use,
— provides well-known annotation formats.

If these annotation tools are to be open-source, they will be found mainly on the website GitHub. This website allows users to rate open-source projects by giving them a star : the more stars a project has, the more adequate it seems. The choice of an annotation tool will therefore be made based on the number of stars, its ease of use and its functionalities. Among the dozens of annotation tools, three were selected based on the criteria stated above : LabelImg, labelme and Make Sense.

LabelImg LabelImg is an open-source annotation tool created in Python language, posted on GitHub where the tool has received more than 17,400 stars. The installation is more complicated than labelme and Make Sense because it is necessary to individually install certain features on the Anaconda software (distribution software for the Python and R programming languages). In addition, it is necessary to download the source code manually via GitHub. Once the software is launched using the Python script, it is easy to use and allows you to easily create and modify annotations, whether existing or not. This tool only supports the YOLO annotation format and thus only allows you to annotate objects in an image using bounding rectangles.

labelme Like LabelImg, labelme is an open-source annotation tool created in Python language, posted on GitHub where the tool has received more than 8500 stars. The graphical interface of labelme very similar to LabelImg suggests that the two are correlated. Labelme is very easy to use and install if the user has the basics of using Python and Anaconda software (distribution software for the Python and R programming languages). In the command prompt, after installing labelme with the "pip" command, simply write "labelme" to launch the software. Once launched, the software allows you to modify and create annotations in the COCO format. Annotations can be made by segmentation or rectangles and can be modified very easily by using a keyboard shortcut when creating the annotation or after creating it.

Make Sense Make Sense is a website for annotating images. The source code of this website is available on Github, has 2300 stars and was developed in TypeScript. To make it work, simply go to the URL "makesense.ai" and follow the instructions. The annotation is less intuitive than labelme because it is necessary to choose to annotate all the objects present in the image in rectangle or in segmentation while labelme offers this hybrid mode. Editing polygons (segmentation) is more complicated : it is more complicated to correct your mistakes during and after creating an annotation. On the other hand, the tool allows you to modify and create databases in multiple formats such as COCO, YOLO or the original .xml format of the Pascal VOC challenge. Make Sense also allows you to annotate images using key points such as the joints of the human body. The additional flagship feature of this tool is the built-in artificial intelligence that allows for automatic annotating as multiple annotations are created.

After analyzing these three algorithms, labelme, which is recommended in the human detection study [14], seems the most suitable for annotating images due to its ease of use (if Anaconda and Python are installed on the user's machine) and support for the COCO annotation format.

2.4 Data scraping

In order to create a database for an object detection algorithm, it is necessary to obtain a large number of images containing the objects that we want to detect using artificial intelligence. Creating a database of this magnitude can be done in different ways : either by creating the database from scratch by photographing the desired subjects and objects, or by downloading suitable images from the internet, or a mixture of both. Collecting images through the internet is simpler and less time-consuming than photographing thousands of images but still

takes a lot of time, which can be reduced by an automated method for collecting these images from the internet called *data scraping*.

data scraping is a method of collecting and downloading data from the Internet automatically. This is the method used by search engines to present search results to the user. There are few image banks available in open access that allow this type of query to be carried out without using scraping tools. The best way to collect a large number of images is to download them directly from a search engine such as Google, Bing or DuckDuckGo.

On the other hand, data scraping is a controversial technique due to the legal implications regarding copyright and terms of use. In addition, the use of data scraping can lead to an inadequate use of the resources requested from the servers hosting the websites : automated searches at a frequency of a few fractions of a second will consume much more energy than a search made by a human every minute. Abusing data scraping on a site can be compared to a DoS cyber-attack [1]. This type of attack can lead to material damage on the servers hosting the attacked site. An ethical use of data scraping is to maintain a reasonable request frequency [37]. Furthermore, copyright and terms of use requirements should not be problematic due to the non-commercial nature of this research work.

Due to these various disadvantages, the use of data scraping is increasingly restricted across the web. An example of such a restriction is Google, which prevents data scraping on its search engine : using such an algorithm on the site will return that Google does not have an image related to the search, while a classic search proves the opposite. On the other hand, the search engine DuckDuckGo [2] is not closed to this type of practice and also provides a tool for this type of query. In order to deliver relevant results, DuckDuckGo obtains its results via more than 400 sources, the main one being Bing [39]. In fact, performing data scraping on the Bing search engine would be equivalent to collecting the same information or images as on DuckDuckGo.

1. *Denial of Service* (DoS) is a type of cyber attack in which the attacker seeks to make the attacked services inaccessible to any other user. This attack is often carried out by flooding the network by sending large quantities of superfluous requests.

2. DuckDuckGo is an alternative to classic search engines such as Google or Bing. Its innovation is to provide additional confidentiality compared to its competitors regarding personal data [38].

Chapitre 3

Object Detection Systems

This chapter is dedicated to the presentation and comparison of existing object detection systems (or algorithms). In order to effectively compare these algorithms, it is first necessary to determine how and on the basis of which measurements to compare them. The machine learning models YOLOv5 and NanoDet will then be presented. The MediaPipe-derived algorithm for human body limb detection created for this work will then be explained as well as the most relevant parameters leading to the optimal results for this system. The three different systems will finally be compared after being tested on the same database presented in the previous section.

3.1 Metrics

The metrics used to compare the different algorithms are the metrics provided by the COCO database, more specifically the *mean Average Precision* (mAP) metric. This metric is the most commonly used for measure the accuracy of detections [40, 41]. In order to clarify the method of calculating mAP, it is important to explain some concepts such as the confusion matrix and the *glsgls-iou* (IoU).

3.1.1 Confusion Matrix

When an algorithm is to predict a value, four possibilities arise from the combination of the algorithm's estimate and the actual value. A true positive (TP) is defined as the estimate and the classification are positive. A false positive (FP) exists when the estimate is positive while the classification is negative. A true negative (TN) is defined as the estimate and the classification are negative. A false negative (FN) exists when the estimate is negative while the classification is positive. The combination of these four possibilities in a matrix is called a confusion matrix [42], shown in figure 3.1.

From the information defined in the confusion matrix, it is possible to extract the values of precision (*precision*) and recall (*recall*). Precision is defined by the true positives divided by the set of predicted positives (true positives and false positives) and allows to determine how many selected elements are relevant. Recall is defined by the true positives divided by the set of actual positives (true positives and false negatives) and allows to determine how many relevant elements are selected. Generally speaking, precision decreases as recall increases during the

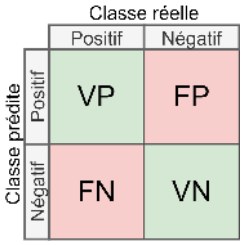


FIGURE 3.1 – Confusion matrix for an object category

training of a system : gradually, more and more relevant elements are selected (recall) while fewer and fewer selected elements are relevant (precision) because, in addition to increasing true positives, false positives also increase. These two parameters can thus be expressed as follows [42] :

$$\text{Precision} = \frac{VP}{VP + FP} \qquad \text{Reminder} = \frac{VP}{VP + FN} \tag{3.1}$$

In the case of object detection, the confusion matrix presented in figure 3.1 is only valid for a single object class. Figure 3.15 in section 3.5.4 represents a confusion matrix defined for several classes.

3.1.2 Intersection over Union (IoU)

The concept of confusion matrix seen in the previous section assumes the binary state of a detection : it can be correct or incorrect. This value is obtained using the *Intersection over Union* (IoU), translated as intersection over union. This measure is defined as the overlap area between the predicted and actual bounding rectangle divided by the union area between these two bounding boxes (see figure 3.2)[40]. Intuitively, IoU represents the correct proportion of the system's estimate.

Using intersection over union, it is possible to determine whether an estimate made by the algorithm is correct relative to a predefined threshold. If the value of IoU, ranging from 0 to 1, is greater than this threshold, the estimate will be defined as correct and vice versa [43]. Using a higher IoU threshold to analyze the results of an object detection algorithm allows to give more importance to the accuracy of the bounding rectangle an object relative to the classification of the detected object.

3.1.3 Average Precision (AP)

By analyzing a confusion matrix (figure 3.1), a system will be more efficient if the true positives and true negatives are much more numerous than the false positives and false negatives. For a system to be efficient, according to the equations (3.1), it is therefore necessary that the precision and recall are both high. By plotting the precision-recall graph in figure 3.3, it is possible to determine the performance of a system by measuring the area underlying the precision-recall curve : the larger this area, the more efficient the system is because the precision and recall will both have a high evaluation. The value of this area is called *Average Precision* (AP) [40][41], which is used to qualify object detection algorithms by category. Since

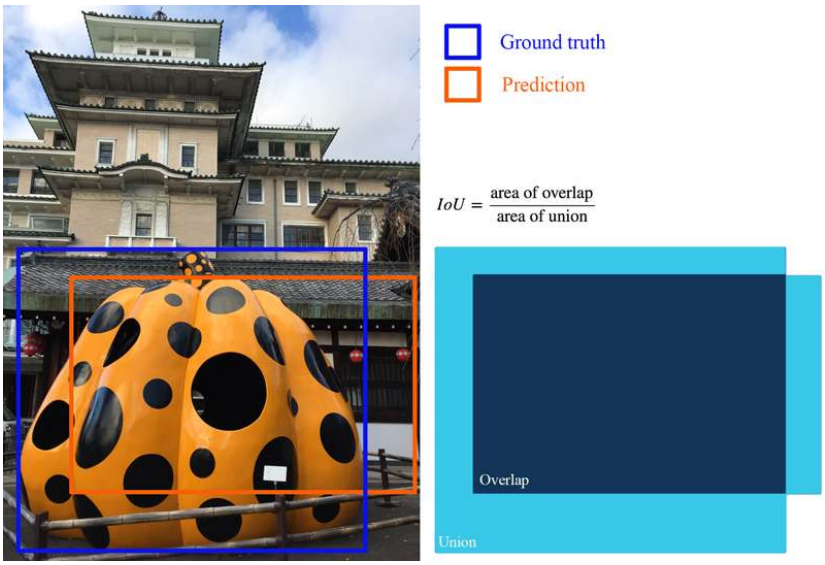


FIGURE 3.2 – Definition and example of *Intersection over Union* (IoU)[43]

the binary state of object detection is needed to evaluate precision and recall, a threshold of IoU needs to be defined to be able to determine the Average Precision.

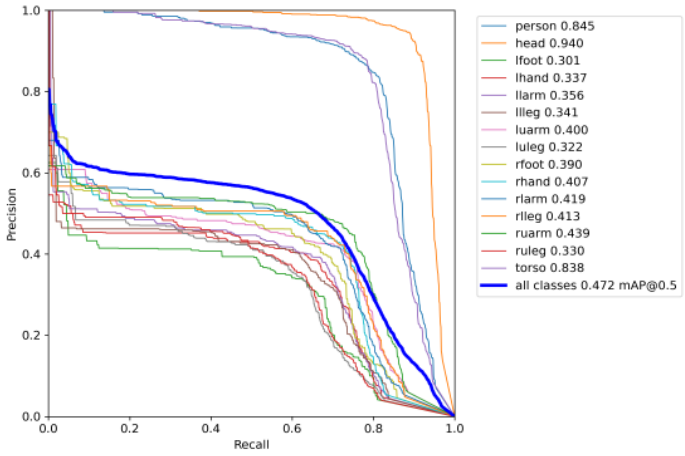


FIGURE 3.3 – Example of a precision-recall curve to determine the *Average Precision*

The Average Precision metric is valid for a single class only. In order to represent the performance across all classes, the *mean Average Precision* (mAP) metric is used. This metric averages the Average Precision of each class [40][41]. In figure 3.3, the thick line represents the mean Average Precision while the thin lines represent the Average Precision for each object category. Since precision and recall are two values between 0 and 1, the values of Average Precision, and therefore mean Average Precision, will be between 0 and 1. The higher these metrics are, the better the system performs.

In order to compare object detection systems using the COCO database, the creators of this database provide evaluation tools [44]. The tools offer six variants of the Average Precision [5] metric :

— Different IoU thresholds are proposed : 0.5, 0.75, and from 0.5 to 0.95 in steps of 0.05 ;
— AP can be measured in different areas : for small objects, the area should not exceed 32 square pixels ; for large objects, the area should be greater than 96 square pixels ; for medium-sized objects, the area should be between the two previous thresholds.

COCO mainly uses the mAP metric for which IoU is set from 0.5 to 0.95 in steps of 0.05. The area-dependent Average Precision metric uses these IoU values. By using multiple IoU values in this way, COCO not only averages the Average Precision over all classes but also over the defined IoU thresholds. In contrast, YOLO uses the mAP metric with a fixed IoU value of 0.5.

3.2 YOLO

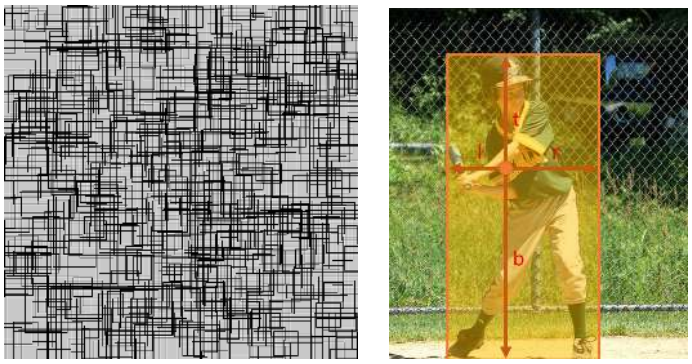
YOLO (You Only Look Once) is an object detection algorithm created in 2016 by Joseph Redmond. This machine learning model is the first object detection algorithm classified as a one-stage detector based on a convolutional neural network (CNN). Before YOLO, object detection algorithms were done in two stages : a first detection stage followed by a second verification stage. YOLO uses a different method : a single convolutional neural network (CNN) is used on the entire image. This CNN divides the image into different regions and simultaneously predicts the bounding boxes and the probability for each region [12]. However, compared to two-stage detector algorithms, YOLO suffers from lower object localization accuracy. This problem has been addressed several times in subsequent versions of the algorithm in addition to improvements in speed and accuracy using various refinements in the algorithm architecture [4, 12, 45, 46, 36]. For the realization of this work, the version of YOLO used is version 5, created and maintained to date by Ultralytics [36]. Version 5 of YOLO has a scalable architecture according to certain parameters so that the user can choose between the nano, small, medium, large and extreme versions in order to best accommodate the use of the algorithm according to the available computing resources. The precision required for the objective of this project and the operational power of the computers available allow the use of the "large" (yolov5l) and "extreme" (yolov5x) versions of the algorithm according to the available graphics memory.

Ultralytics offers some tips to improve YOLO training results. It is recommended :

— to have more than 1500 images per category ;
— to have more than 10,000 instances (annotated objects) per category ;
— that the images are varied ;
— that each instance of each category is annotated ;
— that the annotation is precise ;
— that between 0 and 10% of images are unannotationed. Having blank images helps reduce the amount of false positives.

3.3 Nanodet

Like YOLO, NanoDet is a machine learning model classified as a fully convolutional one-stage object detector (*fully convolutional one-stage object detector*). NanoDet differs from YOLO by not using anchor boxes to detect objects [47]. This type of object detection algorithm was created in 2019 [48] and solves object detection by pixel-wise prediction, similar to semantic segmentation. Anchor box systems detect an object by creating a multitude of boxes in an image to detect whether an object is present in that anchor box (see figure 3.4.a). Anchor box-free systems check whether a pixel under study is part of an object. From the pixel location, a four-dimensional vector determines the size of the bounding rectangle object (see figure 3.4.b).



a. 1

FIGURE 3.4 – Comparison of object detection methods with and without anchor boxes

NanoDet is advertised as a very lightweight and fast object detection model that can perform real-time object detection on small processors such as smartphones. NanoDet claims to be low-power and very accurate. Like YOLOv5, the system has a scalable architecture : it is possible to choose between four different combinations of models thanks to the resolution parameters (320 square pixel or 416 square pixel images), and an architecture multiplication factor of 1.5 or 1.

3.4 MediaPipe

MediaPipe is a framework [1] created by Google in 2019 that allows any developer to select and develop learning algorithms and models, build a series of prototypes and demonstrators, balance resource consumption and solution quality, and identify and mitigate problem cases [50]. Currently, MediaPipe offers computer vision-centric solutions, including face detection, pose detection (using the BlazePose model [21]), hand detection, and other object detection, as well as hair, human body, and object segmentation. These solutions are deployed on different platforms (Android or iOS) and programming languages such as Python, JavaScript, and C++ [51]. In this work, MediaPipe is used for human detection in images using the pose detection tool. Hand and face detection tools have been added to achieve more accurate results. However, this framework does not allow detecting multiple humans present in an image.

1. Frameworks are software developed and used by developers to create applications.[49]

3.4.1 Body Part Detection

Body part detection is performed using MediaPipe's skeleton detection. In motion capture, the human being is represented using point clouds representing key points of the human body such as joints, and segments connecting these points, similar to bones. Together, these segments and points form a skeleton defining the pose of the corresponding human being [18][19]. The skeleton of a human being resulting from MediaPipe's analysis (available in figure 3.5) allows to associate the body parts with their numerical index. Thanks to these numerical indices, it is possible to define the different limbs of the human body : the right forearm will be a combination of indices 14 and 16 which represent respectively the right elbow and the right wrist, the upper part of the left leg will be represented by indices 23 and 25, etc. The different body parts will therefore be annotated using the smallest rectangle delimited by the coordinates of these points of interest, the sides of the rectangle being parallel to the sides of the image.

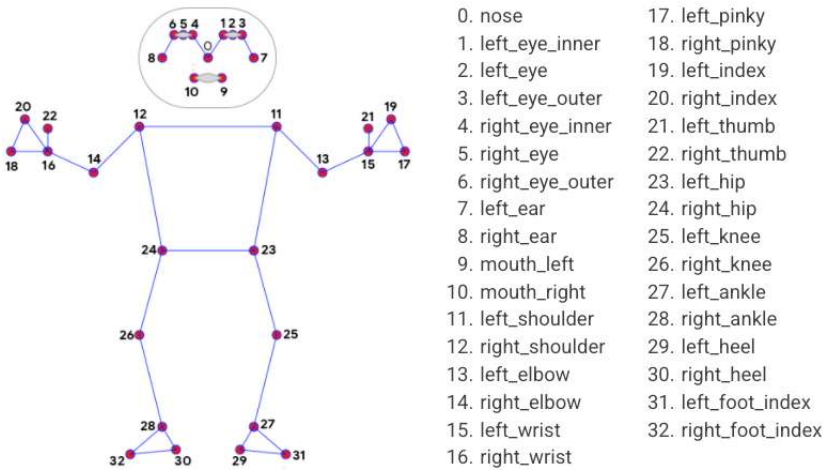
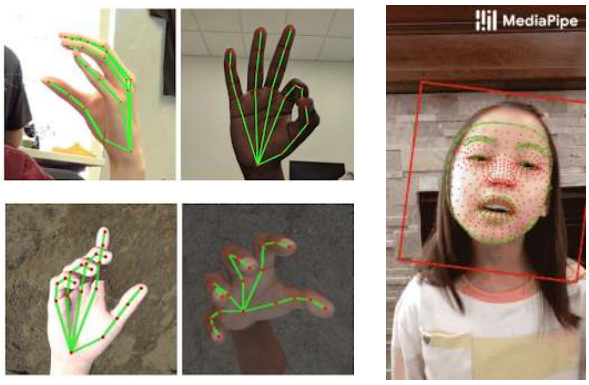


FIGURE 3.5 – Type skeleton generated by MediaPipe and its digital associations to body members [51]

Determining the limbs of the human body in this way is not perfect : if the line segment resulting from the combination of the two points of interest were to be perfectly vertical or horizontal, the bounding rectangle of the limb of the body would then have an extremely small area, which is not in accordance with reality. A solution to this problem is to define a threshold below which the area of the rectangle cannot be smaller : for the arms and legs, this threshold has been defined as the ratio between the height and the width of the bounding rectangle which cannot be less than one third. Not knowing the orientation of the limb of the body concerned (horizontal or vertical position), this threshold can be defined for the width or the height of the bounding rectangle.

In order to detect the head of a human being, facial cues will be used. However, the cues proposed by MediaPipe in figure 3.5 do not sufficiently delineate the face (and therefore the head). The same goes for hands for which the skeleton only surrounds the palms. To solve this problem, MediaPipe provides techniques for hand detection (*hand tracking*, figure 3.6.a) and

faces (*face mesh*, figure 3.6.a). MediaPipe gathers the methods for skeleton estimation, hand detection and face detection in a single feature called *MediaPipe Holistic* that will be used for human detection and annotation in the pipeline presented in the next chapter. The delineations and annotations of the hands and face are done separately from the skeleton. Simply choose the extreme points of the left hand, right hand and face to define the corresponding boundaries.



has. Hand tracking b. Face mesh

FIGURE 3.6 – Hand and Face Components of the MediaPipe Holistic Pack [51]

MediaPipe is not able to detect multiple people in an image, only one skeleton will be generated per image despite the higher number of humans actually present in the image. This problem is solved by extracting each person individually from the image on which MediaPipe is then applied (top-down solution presented in the state of the art in section 1.2.1). To do this, it is necessary to use an object detection model already trained for human detection. Due to the ease of use and the announced and measured performances (see later in sections 3.5.1 and 3.5.4), the choice of the object detection model fell on YOLOv5. The application of YOLO results in areas delimiting the different people. Once these areas are taken out of context, it is possible to apply MediaPipe to them and extract the skeletons (figure 3.7).

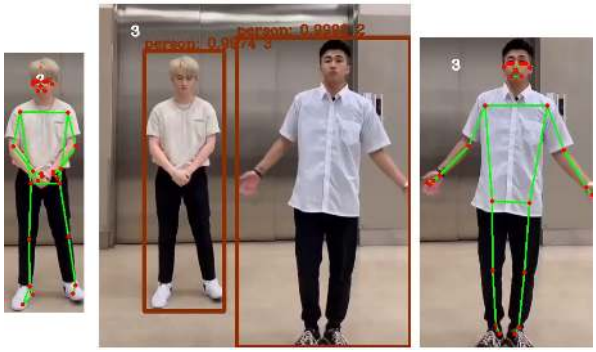


FIGURE 3.7 – MediaPipe applied to multiple people using YOLO [52] (the initial image is in the middle, the images at the extremes are the result of an extraction using YOLO and the application of MediaPipe on this area)

The human body limb detection system based on MediaPipe and YOLO whose person detection is done using YOLOv5 and body limb detection is done by combining different

points of the skeleton estimated by MediaPipe into the smallest bounding rectangle these different points will, after this section 3.4, be abbreviated by the term MediaPipe.

3.4.2 Determination of optimal detection parameters

Each time the algorithms are run on an image, YOLO and MediaPipe provide different trust parameters for their results. The confidence score used by an object detection model indicates the certainty the system has about the accuracy of the predicted rectangle bounding the object and its classification. These trust parameters of the different algorithms inform the user about the certainty the algorithms have in their predictions. The algorithm in question will be completely certain of its prediction if this trust metric has a value of 1 and is completely uncertain if this value is zero. YOLO provides this trust value for each object detected in an analyzed image. MediaPipe displays the resulting skeleton if the trust carried in it is greater than a threshold predefined by the user. In addition, in each skeleton, MediaPipe provides the trust associated with each point of the skeleton.

Before starting the automatic annotation pipeline and starting the downloads and trainings, these three parameters will be predefined in order to have the most correct annotations. In order to determine the optimal values for each of these parameters, each combination will be used to annotate the Pascal-Part database, the results of each combination will then be compared using the evaluation tools provided by the COCO database. In addition to the three trust parameters generated by YOLO and MediaPipe, it is also possible to test the type of model used by YOLO, namely the nano, small, medium, large and extreme models as mentioned in the 3.2 section. The range of the three parameters goes from 0 to 1, this one is divided into three equal parts, each parameter is therefore tested according to the minimum thresholds 0, 0.33 and 0.66. 108 tests were performed to evaluate each possible parameter combination.

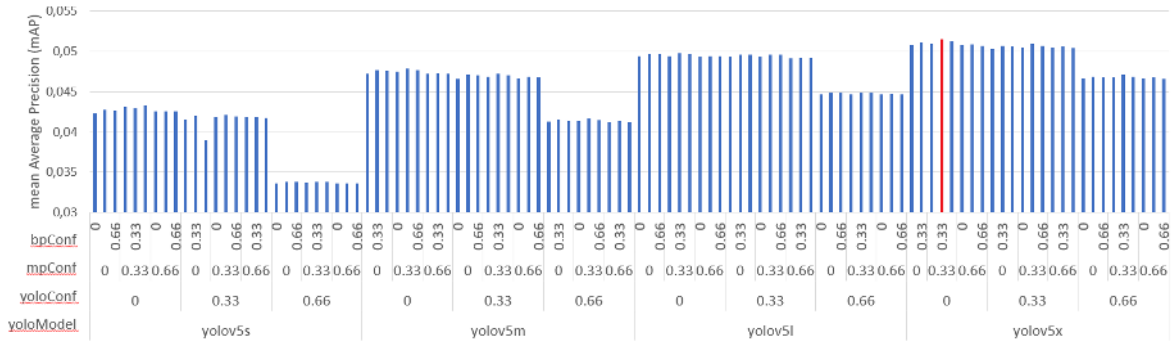


FIGURE 3.8 – mean Average Precision (mAP) of MediaPipe compared to the Pascal-Part database based on different parameters (YOLO model (yoloModel, namely the small (yolov5s), medium (yolov5m), large (yolov5l) and extreme (yolov5x) models), trust thresholds of YOLO (yoloConf), MediaPipe (mpConf) and MediaPipe per body part (bpConf))

The results of the 108 tests (figure 3.8) show that applying to YOLO a trust threshold greater than 0.66 decreases the quality of the annotations : this is due to the lower number of annotations created due to this parameter being set to a threshold that is too restrictive. It

is also possible to observe that the average precision increases with the size of the model : the extreme model (yolov5x) performs better than the small model (yolov5s). Finally, the optimal parameters to adopt that provide the most faithful database annotation possible are (mAP value colored in red in figure 3.8) :

— the extreme model of YOLO (yolov5x),
— a trust index of YOLO at 0,
— a trust index for MediaPipe skeletons at 0.33,
— and a trust index for the different points of a MediaPipe skeleton at 0.33.

Of these optimal parameters, it is interesting to note that the most efficient trust threshold for the YOLO algorithm is 0. This result demonstrates the efficiency of the algorithm because each prediction of YOLO thus leads to an increase in the precision of the annotations : each prediction is correct enough to be annotated.

Among the models proposed by YOLO, the extreme model (yolov5x) is supposed to be the most efficient as announced by the authors of the object detection model. However, It is surprising that the results of this study demonstrate that YOLOv5x is the best model that suits us because of the following analyses. Figure 3.9 compares the detection of people on the same image from the database. The annotations present in the database for this image do not mention individuals present in the image. However, YOLOv5x (figure 3.9.b) managed to detect people, which YOLOv5s (figure 3.9.b) was unable to identify, while the database does not mention them. On closer inspection, it is indeed possible to distinguish people behind the windows of the bus. Thus, YOLOv5x would have made a better detection of human beings in this image than the people who annotated the database. The result of YOLOv5x would therefore be considered false, consequently decreasing the mAP associated with this model. This decrease is however not visible in the results of figure 3.8 due to the uniqueness and rarity of this type of situation in the database.

a. Small model b. Extreme model

FIGURE 3.9 – Comparison of person detection on image by different YOLOv5 models

3.5 Performance of different systems

3.5.1 Announced results

The NanoDet and YOLO machine learning models provide metrics that allow their performance to be compared to other similar architectures. These metrics were calculated based on training of the COCO database and using the associated metric : mAP for an IoU ranging from 0.5 to 0.95 in steps of 0.05. The table 3.1 combines the announced performances of each of the two machine learning models. In its comparison table ([47], not shown here), in addition to the results of the model itself, NanoDet presented the results of the YOLOv3-Tiny, YOLOv4-Tiny and YOLOv5n models, all three of which are scaled-down versions of different YOLO editions. These three models are indeed the most suitable to be compared to NanoDet, the latter being announced as lightweight and suitable for mobile use. By simple analysis of the mAP, it is possible to determine that the YOLOv5 models (except the nano model) are more efficient than those of NanoDet. The reasons for this difference in performance can be multiple : a better training resolution (640 square pixels against a maximum of 416 square pixels), a higher number of training parameters or a more efficient architecture. The power column (expressed in GFLOPS [2]) allows to compare the processing resources required : the NanoDet model is much lighter than YOLO, as announced by the creators of the model.

Model	Resolution	mAP (0.5 :0.05 :0.95)	Power (GFLOPS)	Parameters (in millions)
NanoDet-m	320	20.6	0.72	0.95
NanoDet-Plus-m	320	27	0.9	1.17
NanoDet-Plus-m	416	30.4	1.52	1.17
NanoDet-Plus-m-1.5x	320	29.9	1.75	2.44
NanoDet-Plus-m-1.5x	416	34.1	2.97	2.44
YOLOv3-Tiny	416	16.6	5.62	8.86
YOLOv4-Tiny	416	21.7	6.96	6.06
YOLOv5n	640	28	4.5	1.9
YOLOv5s	640	37.4	16.5	7.2
YOLOv5m	640	45.4	49	21.2
YOLOv5l	640	49	109.1	46.5
YOLOv5x	640	50.7	205.7	86.7

TABLE 3.1 – Comparison of the performances of NanoDet and YOLOv5 announced by their respective creators on the COCO database [47, 36]

3.5.2 Hypotheses

The different algorithms MediaPipe, NanoDet and YOLOv5 were tested to determine which algorithm is the most efficient to detect the different parts of the body after training. It is important to specify that MediaPipe does not benefit from any training but this term is used for convenience compared to the other two algorithms : MediaPipe only detects the

2. FLOPS is an acronym for *Floating Point Operations Per Second*, it is a unit of measurement of the power of a computer corresponding to the number of floating point operations it performs per second. [53] GFLOPS are Giga-FLOPS.

different parts of the body according to the method explained in the section 3.4.1 (inference), therefore without training. The parameters of the NanoDet and YOLOv5 algorithms were set in such a way that the computing resources are used to the best of their capacities on a single Nvidia GTX 1080 TI graphics card. NanoDet was used in its NanoDet-Plus-m version with an image resolution of 320 by 320 pixels and a batch size of 24 images (*batch size*). YOLOv5 was used according to the large model (YOLOv5l), with an image resolution of 640 by 640 pixels and a batch size of 32. Both machine learning models were trained for 200 epochs and all other parameters were left at their default values. The MediaPipe framework is used with the best parameters defined in the 3.4.2 section. The database used is Pascal-Part of which 80

Despite the smaller storage space occupied by black and white images, the Pascal-Part database is used with color images. This choice is dictated by the qualitative analysis of color and black and white photos evaluated with MediaPipe. On each image in figure 3.10 there is a YOLO detection on the left and a MediaPipe detection on the right. The detection using YOLO shows that the algorithm is more confident in its detections on the color image (figure 3.10.b) than on the black and white image (figure 3.10.a). MediaPipe does not detect the man on the far right of the image in the black and white photo but this same man is detected in the color image. This increased performance is explained by the number of additional dimensions available in a color image. A color image has three dimensions to define its colors : each pixel needs three dimensions, one dimension for red, one for green, and one for blue. In contrast, a black and white image, also described as a grayscale image, has only one dimension that defines the gray level on each pixel in the image. For a color image, a machine learning model can then make use of the additional information provided by the additional dimensions that each pixel has at its disposal.

a. Black and white image

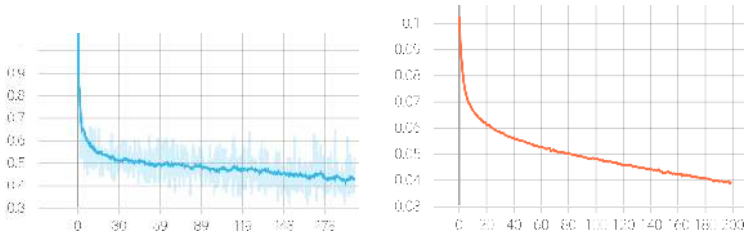
b. Color image

FIGURE 3.10 – Comparison of the qualitative performance of a color image and a black and white image

3.5.3 Cost functions

The graphs in figure 3.11 show the cost functions for the bounding boxes associated with each system tested. Figure 3.11.a shows the cost function of NanoDet and shows that the AI has been trained enough to reach a plateau in the accuracy of localizing bounding boxes of

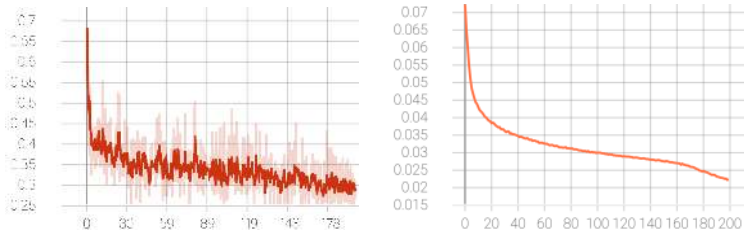
objects : NanoDet would have difficulty reaching higher IoU scores. Figure 3.11.b shows that YOLOv5 has not reached this accuracy plateau but the slope has leveled off : YOLOv5 could therefore potentially be trained on more epochs in order to obtain higher IoU scores.



has. NanoDet b. YOLOv5

FIGURE 3.11 – Cost functions of localization by the number of epochs

YOLOv5 also provides the cost function *cross entropy* plot by the number of epochs (figure 3.12.a). The *cross entropy* cost measures the classification accuracy of each predicted bounding box : each box can contain one class of objects. The equivalent provided by NanoDet is called the *quality focal loss* (figure 3.12.b), and is a combination of classification and object localization. For the same reasons as in the previous paragraph, YOLOv5 could potentially be better trained.



has. NanoDet b. YOLOv5

FIGURE 3.12 – Cost functions of classification by the number of epochs

3.5.4 Test Results

Table 3.2 shows the results of training the different systems using the mAP metric at different IoU thresholds and the respective training times. For each algorithm, the mAP value at an IoU threshold of 0.5 is approximately double the mAP value at an IoU threshold of 0.75 meaning that object detections are evenly distributed around an IoU of 0.75. The mAP value at the IoU threshold ranging from 0.5 to 0.95 in intervals of 0.05 (0.5 :0.05 :0.95) demonstrates the overall performance of the analyzed systems : YOLOv5 performs 1.5 and 5 times better than NanoDet and MediaPipe, respectively. A similar interpretation can be used for the other values of mAP making YOLOv5 the system of choice when training an annotated database. The last row of the table 3.2 shows the performance per unit time, the unit being mAP per hour, the mean Average Precision (mAP) being taken at the IoU threshold 0.5 :0.05 :0.95. A higher value indicates a higher performance in less time. Thus, NanoDet seems to be more efficient than YOLOv5 according to this metric. Given the same time, NanoDet will therefore

be more efficient than YOLOv5, as announced by the creators of the model. The performance per hour of MediaPipe is not comparable to machine learning models because MediaPipe does not train but only detects objects.

IoU	**YOLOv5**	NanoDet	MediaPipe
0.5 :0.05 :0.95	24.8	16.34	5.22
0.5	45.1	30.4	9.11
0.75	24.15	15.08	5.03
Training duration	18.5 hrs	10.2 hrs	2 hrs
mAP (0.5 :0.05 :0.95) per hour	1.34	1.6	2.61

TABLE 3.2 – mean Average Precision (mAP) according to different IoU thresholds for each system and their respective training durations

Comparing each algorithm by body member category in figure 3.13, the AP differs somewhat from the interpretation made using the mAP. An individual is detected in an equivalent manner by each algorithm : MediaPipe has an average precision on this category of 88

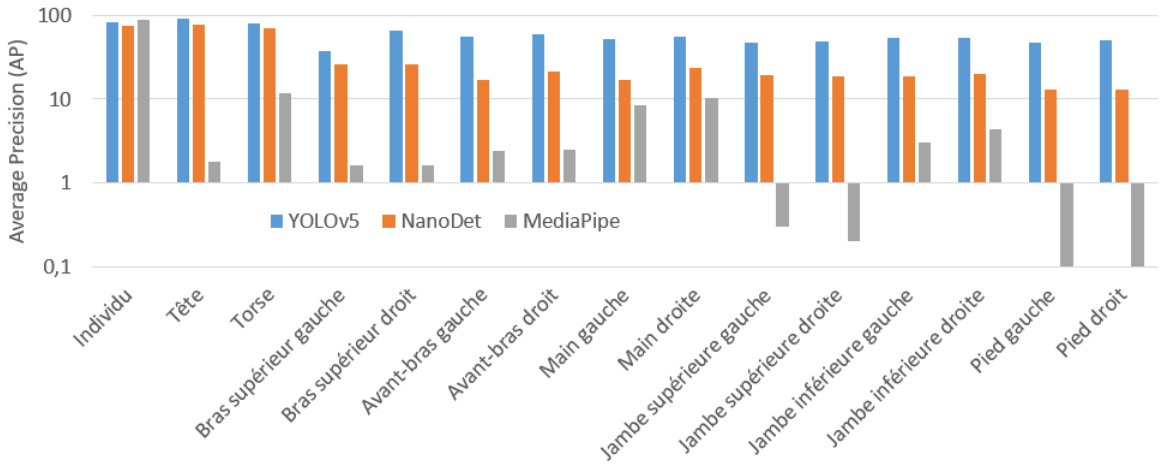


FIGURE 3.13 – Comparison of Average Precision (AP) for each body member category between YOLOv5, NanoDet and MediaPipe

The rest of the categories follow the framework seen in the table 3.2 : Nanodet is still a little less efficient than YOLOv5 but much more efficient than MediaPipe. The MediaPipe algorithm uses three main detection tools : hands, face and skeleton. This algorithm is therefore naturally better in detecting hands. On the other hand, it is less precise in detecting the head. Indeed, the algorithm only detects the face which is only part of the head. This explanation is also valid for detecting feet whose skeleton only has points for part of them.

The lower results of MediaPipe are probably due to the fact that MediaPipe was trained to detect people who are well exposed and well illuminated, and not people in unusual positions in which some limbs might be hidden. Figure 3.14 provides a qualitative comparison between

an image adapted for MediaPipe (fig. 3.14.b) and an image that is not (fig. 3.14.a). Comparing the body parts by color codes, image 3.14.b shows a correct estimate of the skeleton because the situations (exposure, illumination and pose) are ideal, which is not the case for image 3.14.a (photo of a human being in a sporting activity) which wrongly estimates the position of the rider : the feet and legs estimated by MediaPipe are not supposed to be in this location relative to his horse. However, the state of the art has proven that human detection is more complicated when the subject is practicing a sporting activity due to the unusual poses adopted by the subject. Figure **??**.a is therefore an example of an image in which the pose of the human being generates a problem in detecting the subject. The database used to compare the algorithms with each other, including images of this type, explains the inferior results of MediaPipe compared to machine learning models. However, it is important to specify that the results of the image **??**.a do not respect the confidence thresholds established in the section 3.4.2, thus allowing to better illustrate the points discussed in this paragraph.

a. Unsuitable image b. Suitable image

FIGURE 3.14 – Comparison of a MediaPipe-adapted and non-MediaPipe-adapted image

In the results that YOLO provides, there is also a confusion matrix taking into account the studied categories (figure 3.15). Like the confusion matrix for a single category in figure 3.1, the higher the number in a box located on the diagonal of the matrix, the more frequent the association between the real class and the predicted class. The matrix provided by YOLO confirms that the best detected classes are people, torso and head. This matrix shows that the categories are rarely confused with each other : the incorrectly detected elements are usually part of the background of the image. More precisely, these incorrect elements are false negatives (*background FN* in figure 3.15) meaning that the algorithm fails to detect certain objects where they are present. However, this last remark is not correct for symmetrical members of the human body such as arms and hands, and legs and feet. For these categories, it often happens that YOLO confuses a member located on the left side with an analogous member located on the right side of the human body. Unifying the symmetrical categories would therefore allow to obtain better results, at the expense of the left and right detection of the elements concerned.

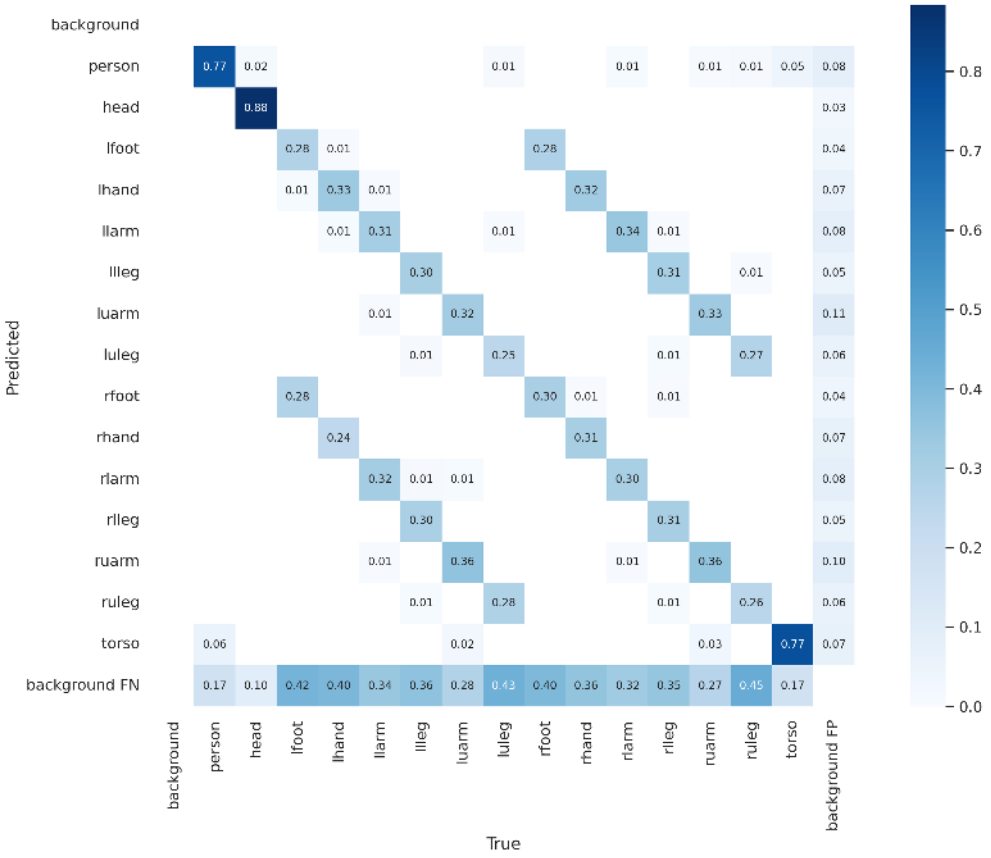


FIGURE 3.15 – Confusion matrix with multiple object categories resulting from the learning of YOLO validated on the Pascal-Part database

3.5.5 Summary of the comparison between the three models

The detection results of the MediaPipe, NanoDet and YOLO object detection algorithms allow us to distinguish between the three : YOLO is a better object detector than NanoDet, which is itself better than MediaPipe. Different tests have shown that the use of color images allows us to obtain better results, as well as the determination of four other parameters inherent to the object detection system created using MediaPipe (YOLO model, YOLO confidence, MediaPipe confidence by skeleton and by body part). Finally, it has been shown that the detection of human beings is more complicated in a sports activity context as confirmed by the state of the art.

Chapitre 4

Automatic Annotation Pipeline

4.1 Overview

Automatic annotations of human body parts are performed using the MediaPipe algorithm presented in section 3.4.1. Figure 4.1 presents the overview of the model architecture for detecting human body parts. This algorithm will be explored in detail in the next section. The first step of this algorithm is to create the database by downloading images using data scraping, images that will then be annotated using the system created based on MediaPipe and YOLO (for multiple person detection). The entire algorithm is called pipeline : in machine learning, a pipeline refers to the steps required to train a model.

Once the database is created, a machine learning model needs to be trained to become a human limb detector. To do this, the database will be trained multiple times, shuffling it at each iteration. These shuffles ensure that the entire database has been trained using a different dataset each time. The weights trained at each iteration are used in the next training to continue the training. In essence, this is a single training of a machine learning model, shuffling the data once in a while to simulate training with a larger database than it actually is. The object detection model chosen is YOLO because of its superior performance presented in section 3.5.4. YOLO expresses its results using a Intersection over Union threshold of 0.5 for the mean Average Precision metric and will be the metric used throughout this chapter with some exceptions.

Once the model is trained on the database created using data scraping, the system will be used on an external and previously annotated database in order to determine the quality of the database created and the annotations created using MediaPipe : in a classic machine learning pipeline, this is the validation step, the external database being the validation database.

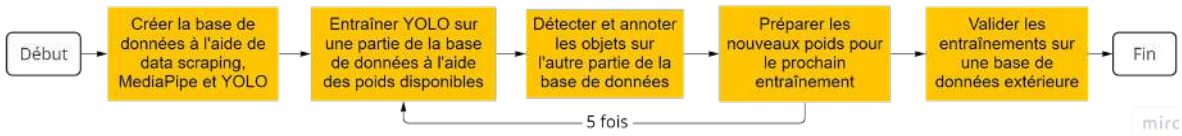


FIGURE 4.1 – Pipeline Overview

This chapter covers :
— of the architecture created and explained,
— of images collected using data scraping and the quality of automatic annotations,
— of optimal parameters to use for the machine learning model in order to get better results,
— and validation of the best model on the external database.

4.2 Architecture

Figure 4.2 presents the architecture for creating the automatic annotation system for human body parts. The architecture starts with data collection following the data scraping method presented in section 2.4. This part collects only a small number of images, for example 20 samples, before annotating them automatically using MediaPipe and YOLO according to the method described in section 3.4.1. This automatic annotation starts with the extraction of individuals in the analyzed image using YOLO. The resulting image parts containing these individuals will be annotated in order to decide if they are good enough to satisfy the parameters defined in section 3.4.2. These last actions will be performed for each new image in the database. Once these annotations are done, it is checked whether there are enough annotated images in the database. If so, the data collection and automatic annotation are executed again until a sufficient number of annotated images are reached or if the collection no longer returns new images.

Once the database is created, it is necessary to prepare the data so that it can be trained with the YOLOv5 model. Compared to the YOLO annotation system, the COCO annotation system was chosen to annotate the data because of the additional information available. However, the YOLO annotation system allows to manipulate the database more easily. Thus, once the database is created, the next step is to convert the annotations for the YOLO system before splitting the database : 30

Once the data is converted to YOLO format and mixed into different datasets, the next step is to train the object detection machine learning model. Training with such a model means that the system must determine the weights assigned to each neuron in its neural network. Since this determination is done by trial and error, training can sometimes take several hours. When training is finished, the weights of each neuron are saved in a suitable file. These weights are used in the next step during inference : this step consists of annotating the images from the inference dataset using the weights trained during the previous training.

The data is then shuffled again. The newly annotated dataset - the inference dataset - becomes the training dataset. The training datasets and the rest are shuffled together. Part of this shuffle is then redistributed into the inference dataset so that the number of samples in this dataset is the same as in the training dataset, i.e. 30

After the database has been shuffled and reshuffled, the pipeline trains YOLO on the new training dataset. The detection, database shuffling, and training steps are repeated an arbitrarily determined number of times, in this case five times. The resulting weights from all trainings

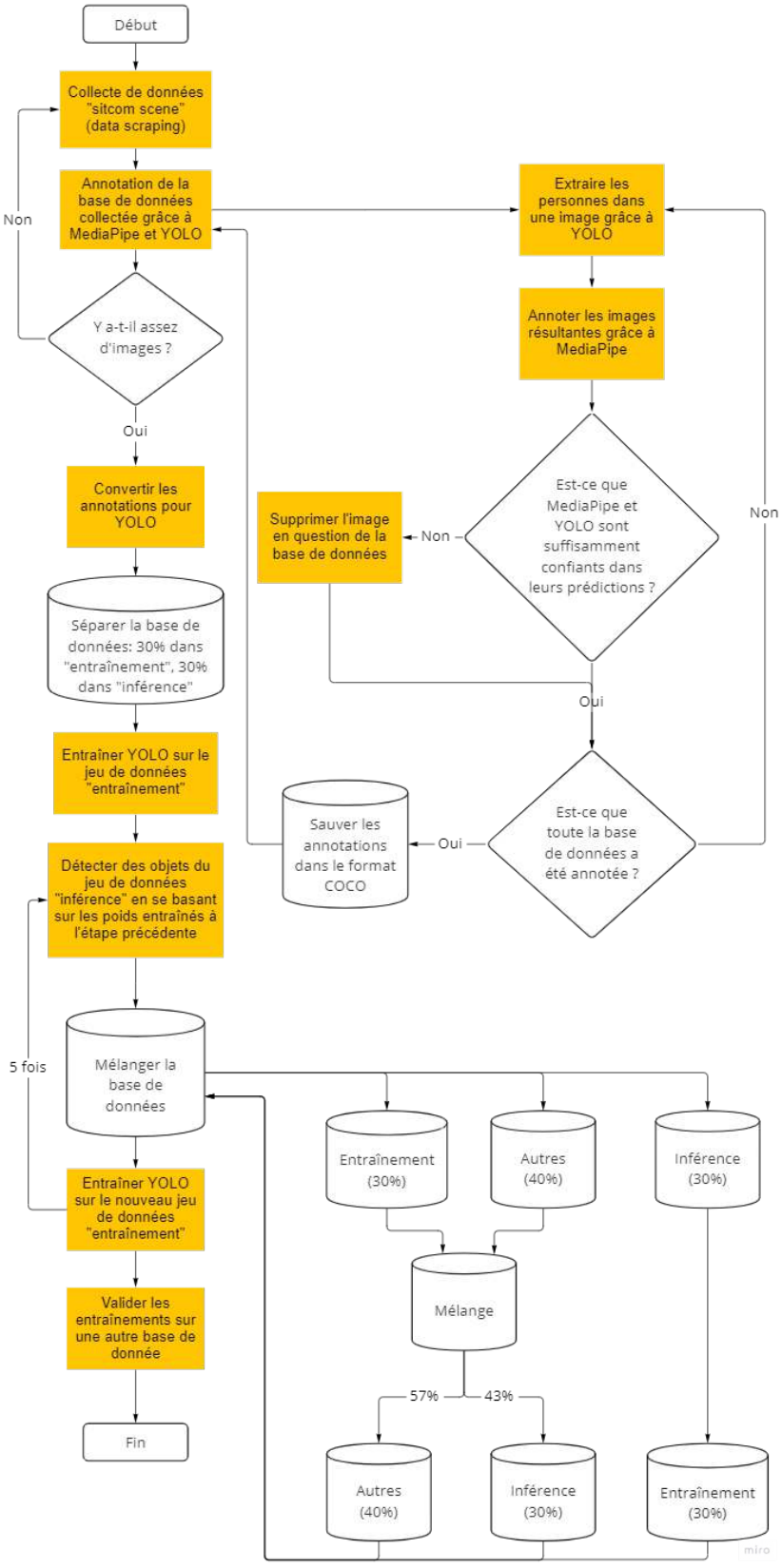


FIGURE 4.2 – Pipeline architecture

performed are finally validated on a manually annotated external database to determine the pipeline's performance.

4.3 Data Scraping

In order to create a database quickly by collecting images from the internet, these images are collected using a data scraping algorithm as presented in the 2.4 section. This data scraping algorithm will not be performed with Google due to the blocking put in place by the search engine towards the method. Bing will not be used either because DuckDuckGo collects the same images and provides tools to do data scraping more easily. Thus, among the Bing, Google and DuckDuckGo search engines, the algorithm aims to collect the images proposed by the DuckDuckGo search engine based on a predefined search term.

The objective of this work being to best annotate the human being in his everyday life, multiple image searches on the different search engines were carried out. It follows from this study that the images best representing humans in everyday life are images from television series of the "situational comedy" genre, also known as "sitcom". A sitcom can be defined as "a television series generally using situational humor and inspired by everyday life" [54]. This definition confirms that the resulting images will have the form of scenes from everyday life and whose lighting of the actors is optimal. The search term used to create our database will then be *"sitcom scene"*. Adding the keyword *"scene"* allows to obtain images of the series while using *"sitcom"* alone allows to obtain promotional posters that are not representative of daily life and are not suitable for the use of this project. A sample of images from the database thus created is available in figure 4.3.

FIGURE 4.3 – Sample of the database resulting from the data scraping of the keywords "sitcom scene" requested from the DuckDuckGo search engine

4.4 Annotation Results

Number of images and distribution of average confidence The data scraping of the keyword "sitcom scene" allows to collect 796 images to annotate for the database. The database thus created will be called "sitcom" in the rest of the document. However, 796 is the

number of images collected in the first collection, this number may change over time due to the very concept of the internet constantly evolving. Subsequent collections made it possible to estimate the number of images present in a collected database at approximately 815 images. It is possible to assign an average trust score to each image by averaging the confidences of each object present in the image (presented in section 3.4.2). Each detected person will thus have an average for the skeleton keypoints, an average for the skeleton detection and an average for the person detection itself. By defining a threshold for the average confidence score, it is then possible to reject images for which the score is lower than expected. The distribution of the average trust applied to all the images collected in figure 4.4 shows that it is at the value 0.83 and that nearly 25

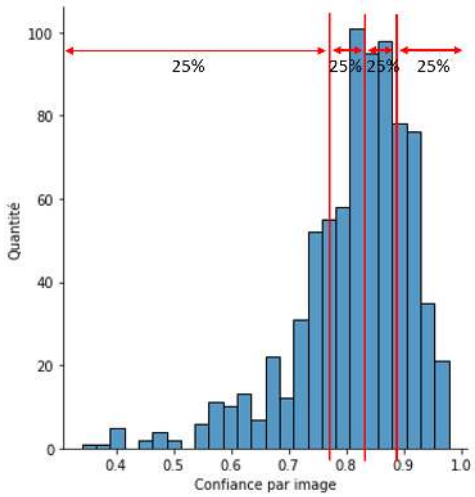


FIGURE 4.4 – Distribution of the average trust of images annotated by MediaPipe and YOLO

Distribution of annotations by category Figure 4.5 shows the distribution of the different annotations for each category of body parts as well as the associated average areas measured in square pixels. The area of an annotation is measured by multiplying the height and width of the rectangle surrounding the associated body part. Figure 4.5 allows us to anticipate the results of the different learning processes to be carried out. It is possible to predict that the detection of people will be easy due to the number of available examples and the associated average area : the category of people is on average present more than four times per image (3149 instances in total) and has an area four times greater than any other category (22591 square pixels). Legs and feet are the least represented in the database and will be more complicated to train. This underrepresentation of the lower body members is due to the choice of the database : in sitcom television series, it is more interesting to film the actor's torso and face when he speaks than his legs. In addition, special attention must be paid to the feet which are the least represented body members (less than 522 instances per foot in total) while having the smallest average area (the average area of the two feet is 265 square pixels, approximately 10 times smaller than the category of people). This underrepresentation suggests that the detection of feet will be less accurate compared to other body members. A rebalancing in order to fairly distribute the number of annotations in each category may be necessary.

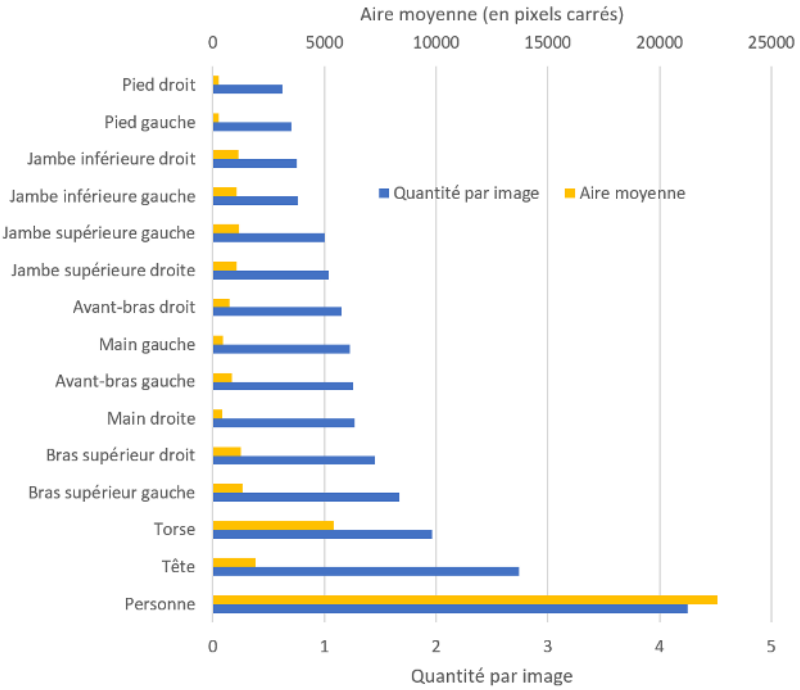


FIGURE 4.5 – Quantity of annotations per image per category and their associated average area

4.5 Comparison of different parameters based on results

The tests in this section for the comparison of the different parameters were performed with the small and large models of YOLO explained in section 3.2, the default model being the small model. The optimal parameters of MediaPipe determined in section 3.4.2 were used. The database used for these tests is the Pascal-Part database.

4.5.1 Minimum Average Confidence Score

As seen in the previous section, the minimum average confidence score allows to reject images whose annotations are not of sufficient quality and precision. Using the experience acquired during the various tests, this threshold was arbitrarily set at 0.6. This practice allowed to eliminate 55 images from the sitcom database, or 7Subsequently, training was performed with a minimum mean confidence threshold of zero and compared to a threshold set to 0.6.

Figure 4.6 shows the comparison between the training of the pipeline for which the minimum average confidence threshold was set to 0.6 (in yellow) and 0 (in blue). In general, when no threshold is set, the results expressed in Average Precision are more evenly distributed across the different categories. This results in a mean Average Precision score that is 8

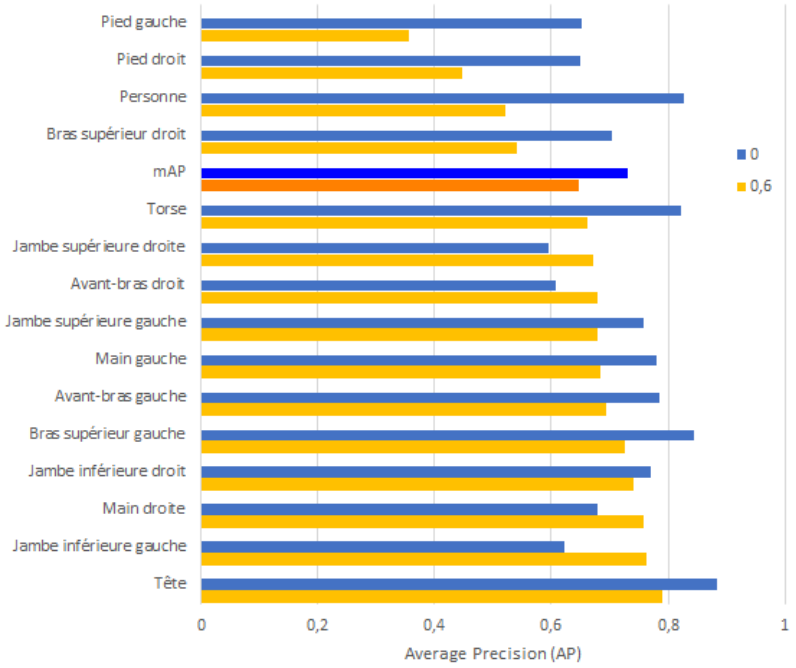
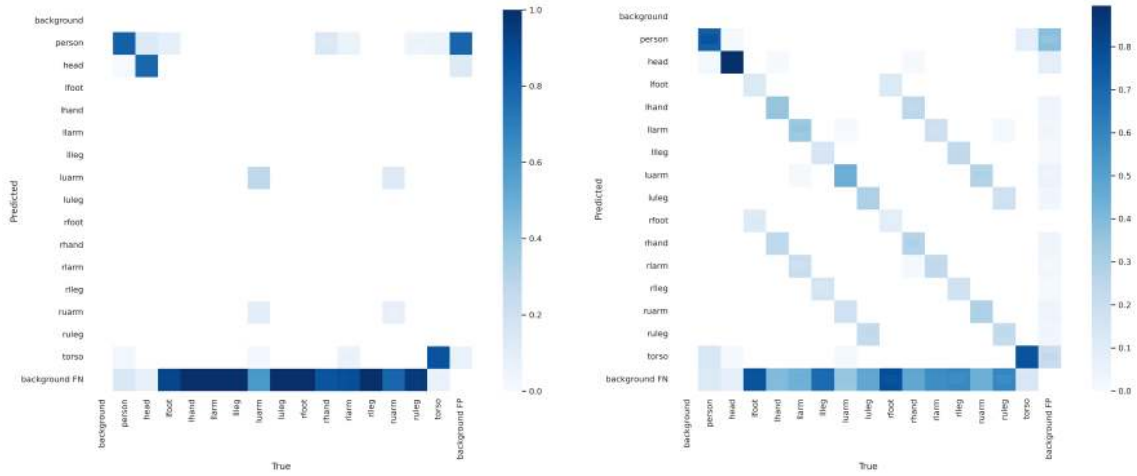


FIGURE 4.6 – Comparison of Average Precision score by category based on selected average confidence threshold

4.5.2 Training using pre-trained weights

By default, YOLO provides pre-trained weights on the COCO database comprising 80 categories including the people category and whose mean Average Precision score (0.5 :0.05 :0.095) is between 28% and 50.7% depending on the object categories. It was deduced that the created algorithm detects body parts once the people are detected. Running the automatic annotation algorithm using these weights should lead to better results because the people category is already trained. This category being trained, the machine learning model will only have to detect people to be able to detect their parts (arms, legs, torso, etc.).

Figures 4.7 show the confusion matrices of an algorithm trained with or without prior weights. The assumption made in the previous paragraph is true : body part detection is much better with the trained weights. Training without pre-trained weights still managed to detect and train the categories of people, head and torso : these categories are indeed the most represented in the database according to figure 4.5 (when the average confidence threshold is zero, as is the case for this test) and have the largest surface area of all categories. The loss of information from body parts other than the head and body occurs in the pipeline from the first training of YOLO. At this stage, the machine learning model starts by training the most represented and therefore easiest categories : people, heads and torsos. At this stage, i.e. at the end of the first iteration, the other categories are also trained and the resulting detections are better than in the final iteration shown in figure 4.7.a. However, these resulting detections are not good enough to be trained in subsequent iterations. Indeed, in the pipeline, the following iterations use the annotations detected by the weights trained in the first iteration. Since these



a. No weights b. Weights provided by YOLOv5

FIGURE 4.7 – Confusion matrices as a function of the weights used to train the YOLO system on the sitcom database

detected annotations are not of sufficient quality for the following iterations, they are lost as the training and pipeline progress.

Finally, the lines parallel to the diagonal of the matrix in figure 4.7.b testify to the confusion of the pipeline for the detection of left and right human body limbs : the pipeline confuses a left limb with a right limb.

4.5.3 Choosing the YOLO model

As presented in section 3.2, YOLO has multiple models that allow to increase the accuracy of the model at the expense of the consumption of computing resources. The computers used for the pipeline training have Nvidia GTX 1080 Ti or Nvidia GTX Titan V graphics cards. On either graphics card, the large model (yolov5l) is the most advanced to learn to detect body parts without exceeding the capabilities of these graphics cards. Figure 4.8 presents the training results using the small model (yolov5s) in blue and the large model (yolov5l) in yellow, on the same database, without a minimum threshold of average confidence (threshold at 0) and using pretrained weights.

Figure 4.8 shows us that the mean Average Precision of the large model obtains a score of 78.8

4.6 Algorithm validation

Once the best parameters of the automatic annotation algorithm have been defined, the resulting model can be used on an external database to determine the accuracy of these automatic annotations : this is the validation step of machine learning. If the algorithm was able to correctly detect objects on the validation database, there is a good chance that the automatic annotations will be correct on other databases. In our case, the test was performed on

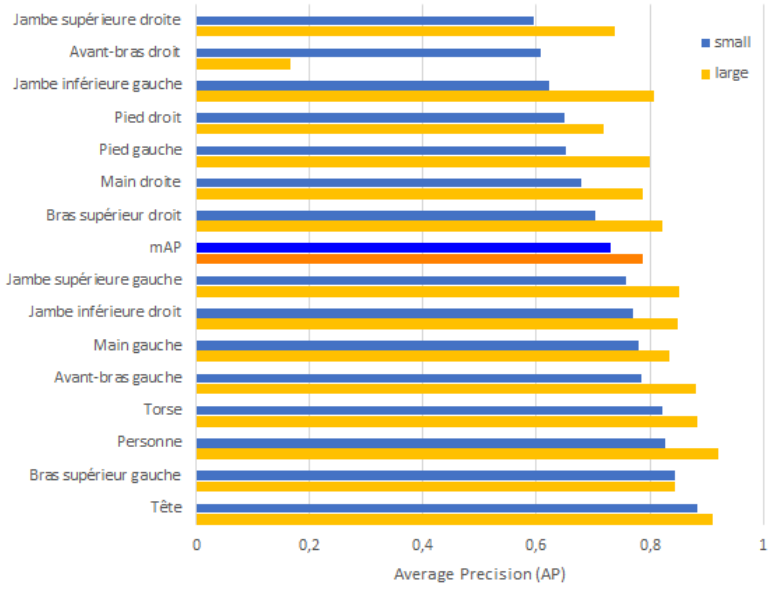
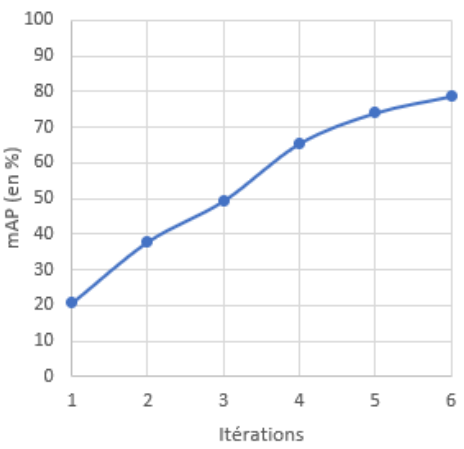


FIGURE 4.8 – Average Precision scores per category as a function of the size of the model chosen to train the YOLO system on the sitcom database

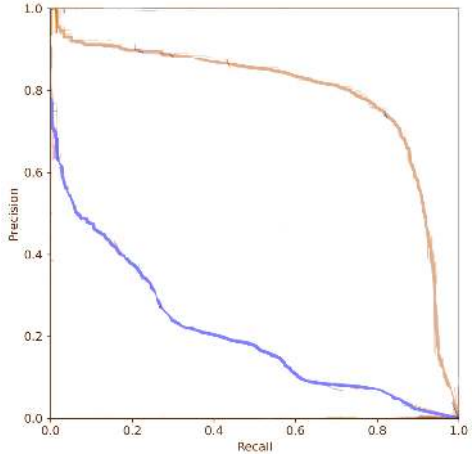
the database collected using the data scraping "sitcom scene". The more varied this database was, the better the machine learning model trained on it will be able to annotate another database : the database chosen to perform this validation is the Pascal-Part database. The YOLOv5 model for people detection chosen is the large model presented in the 4.5.3 section which improves by 14.1

4.6.1 Performance Evolution by Iteration

Before performing these tests, it is useful to certify that the machine learning model has been sufficiently trained by checking whether each cost function of each iteration ends up stagnating as explained in the 3.5.3 section. During the first iteration, all the cost functions stagnate once the epochs are performed. Furthermore, the following iterations do not allow for further evolution compared to the first iteration : the system stops training prematurely thanks to the early stopping which allows to stop this training if no evolution of the mAP is observed during a defined number of epochs. These following iterations actually undergo saturation and the mean Average Precision decreases rapidly after the first epochs. Since the algorithm only records the best results, figure 4.9.a shows the evolution of the mAP as a function of iterations. Figure 4.9.b shows the precision-recall curve of the first (in blue) and last (in orange) iteration on which the mean Average Precision score is based (see section 3.1). The area under the blue curve is much smaller than the area under the orange curve, confirming that the mAP has increased between the first and last iterations.



a. Evolution of mAP by iteration



b. Precision-recall curves of the 1st (orange) and 6th (blue) iteration

FIGURE 4.9 – Improvement of mAP as a function of the number of iterations

4.6.2 Validation using the Pascal-part database

Figure 4.10 shows the confusion matrix of the Pascal-Part database applied to the model and weights chosen to validate the algorithm. The result mAP is 10.1% and corresponds to the confusion matrix : the algorithm struggles for all categories. The easiest categories to detect, i.e. people, heads and torsos, are not sufficiently represented, with heads being more difficult to detect than the other two categories (people and torsos). It would seem that for the algorithm to detect body parts, it must first detect people, but these are poorly represented. The category of people is very often detected where it is not present : there is a large quantity of false positives (right column *background FP*) for this category.

These results are a typical example of the overfitting phenomenon. overfitting occurs when the machine learning model has learned the images used for training too precisely and is not able to adapt to images that come from a database other than the training database. In this case, the images in the sitcom database are too different from the Pascal-Part database. The sitcom database is adequate to represent well-lit human beings in an everyday environment : the images are "too perfect" compared to everyday life where lighting and framing are not always perfect. In real life, everyday life is not always well lit or framed and does not exist in a movie studio. We can therefore conclude that the sitcom database is not suitable for detecting human beings in varied images. On the other hand, using it would surely be effective when the images are similar to sitcom images.

The presence of a large number of false positives in the people category is also a consequence of overfitting. Figure 4.5 informs that on average four people are present per image in the sitcom database. However, the Pascal-Part database contains a large number of empty images, without annotation, the latter being initially devoted to other categories that were removed from this case study in order to facilitate learning (such as planes, bicycles, birds, etc.). Since each image in the sitcom database has at least one human being, the model creates a lot of false positives because it thinks that a human being will automatically be present in an image,

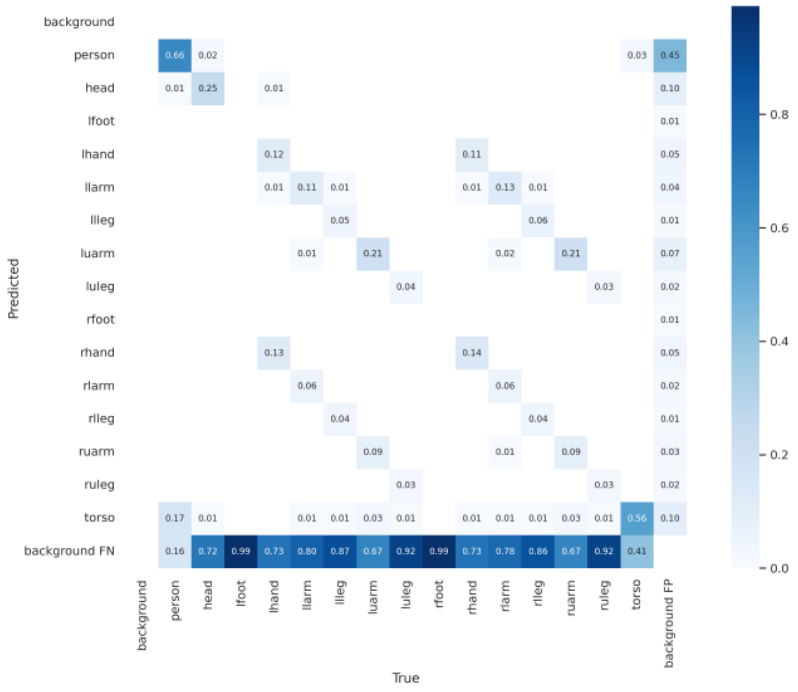


FIGURE 4.10 – Confusion matrix resulting from the validation of the automatic annotation algorithm

which creates erroneous annotations.

4.7 Training on another database

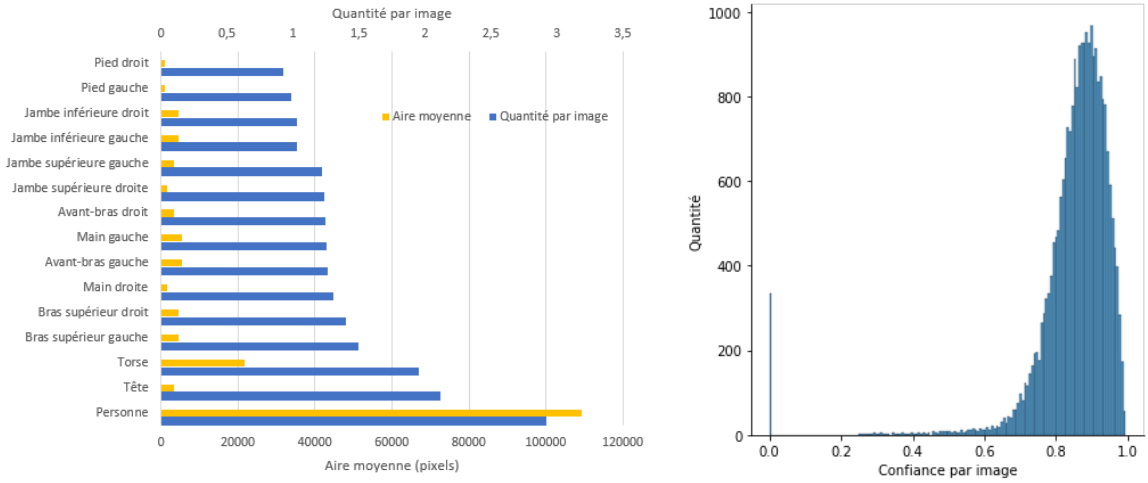
The previous section showed that the poor performance of the created automatic annotation algorithm is probably due to overfitting. This phenomenon can be reduced in two ways : limiting the number of epochs or increasing the amount of images in the database. The sitcom database with only about 800 images can be considered as incomplete. Therefore, new keywords must be found to collect more images using data scraping or a new data scraping method must be found. The *MPII Human Pose Dataset* database mentioned in section 2.1 was collected via videos on the YouTube website using different keywords and thus collected 25,000 images. For ease and time saving, the automatic annotation pipeline will be trained using this database in order to simulate training on a database created via data scraping on YouTube. For the sake of time, the data scraping tool on YouTube has not been created but the development of such a tool is perfectly feasible [55]. The first results presented were achieved by distinguishing between the left and right limbs of the human body. In order to find avenues for improvement, the second results presented were achieved without distinguishing between the left and right limbs in order to group the analogous categories into a single category.

4.7.1 Results with distinction of left and right members

The algorithm was not trained with the best model presented in the previous sections but with the YOLO small model (yolov5s) because training with the large or extreme model would

have taken several days, or even more than a week on a database of such size. The training database is *MPII Human Pose Dataset*, abbreviated to MPII in the rest of the document.

Figures 4.11 present the statistics of the MPII training database. Figure 4.11.a shows that the distribution of categories is more homogeneous than in the sitcom database : the ratio between the number of annotations per most and least present category is higher for the sitcom database than MPII. The confidence score per image for the MPII database (figure 4.11.b) is high : the average score is 0.82. This graph also shows that about 350 images are images in which no human being was detected, this characteristic was not present in the sitcom database.



a. Quantity of annotations per image per category and their associated average area

b. Distribution of the average confidence score

FIGURE 4.11 – MPII Database Statistics

The mean Average Precision score of the MPII database is 72.4

The validation results using the Pascal-Part database of the automatic annotation algorithm trained on the MPII database improve slightly compared to the training with sitcom. The mean Average Precision is at 14.1%, 4 percentage points higher than the previous validation test, and the confusion matrix in figure 4.13 shows that the detection of small objects remains complicated.

These results can be explained in two ways : either the training suffers from overfitting, or the model needs to be improved. If the overfitting can be reduced by increasing the database or decreasing the number of epochs and if the database is large enough, it may be interesting to validate the model using the weights from the training of previous iterations. Validating the algorithm using the weights from the first training iteration does not offer any improvement with a mAP score of 10.9%. Using the weights from the fourth iteration, i.e. before losing the detection of the right foot, the mAP result is at the same level of mAP as with the weights from the last iteration and therefore does not offer any improvement. In this specific case, the

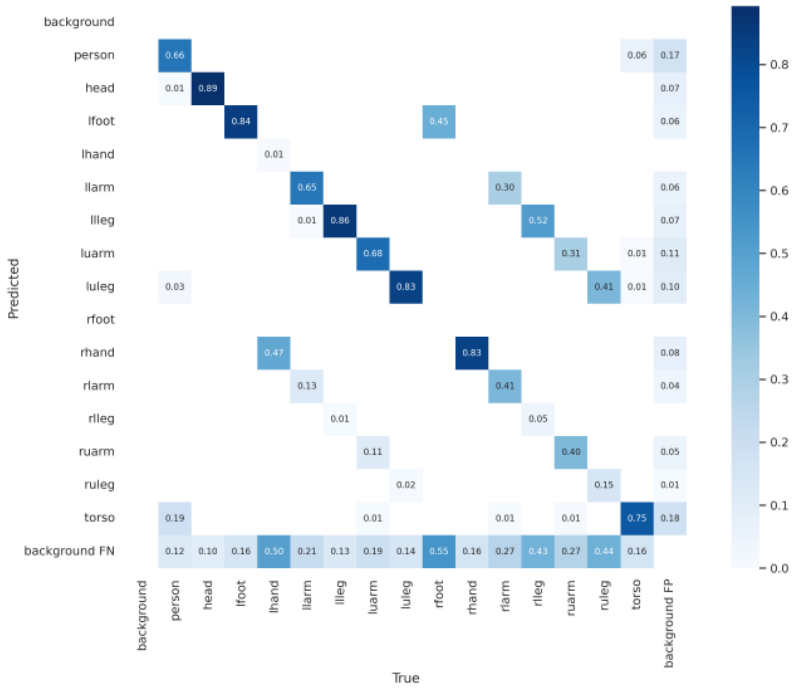


FIGURE 4.12 – Confusion matrix resulting from the validation of MPII based on MPII training weights

fourth iteration can therefore be considered the best iteration. Generally speaking, the last iteration is considered the best iteration.

4.7.2 Results without distinction between left and right members

Distinguishing between left and right limbs opens the door to more errors when training the machine learning model due to the increased number of object categories and the possible confusion between analogous limbs of the human body. Not making this distinction should thus lead to improved training and validation results. The training results using the MPII database without making this distinction are presented in the form of the confusion matrix available in Figure 4.14. The mAP score with a IoU threshold of 0.5 is 83.7

When the model that does not distinguish between left and right limbs of the human body is validated on the Pascal-Part database, the mAP score obtained is 17

These validation tests were performed on the entire Pascal-Part database, which includes images containing people and images without annotation (no people present in the image). Since the model learned to identify humans on the majority of the images received, validating the pipeline using the subset of the database containing only images containing humans reduces the number of false positives generated by the model. The results of such a validation provide a mAP score of 19.7

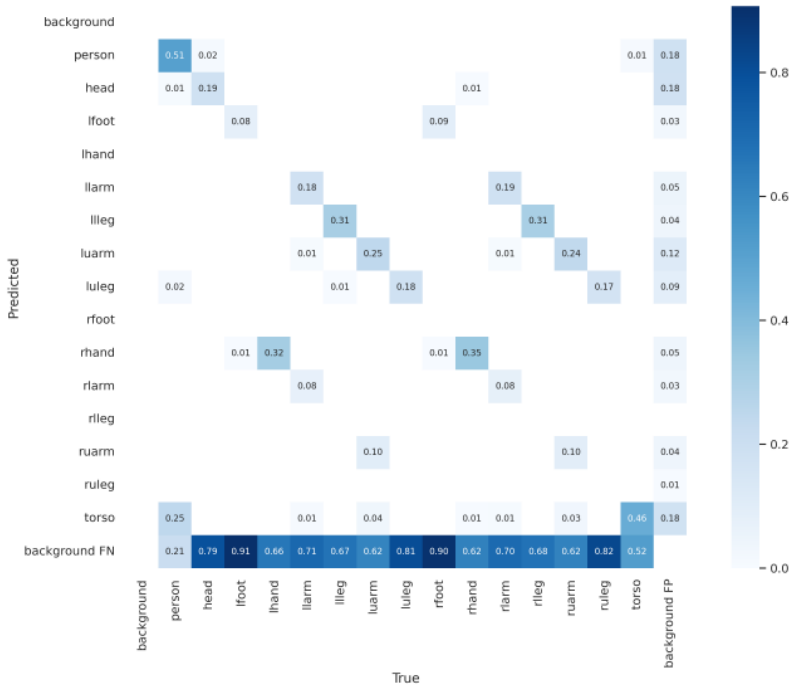


FIGURE 4.13 – Confusion matrix resulting from the validation of Pascal-Part based on MPII training weights

4.7.3 Review of object annotation and detection results using the pipeline

The pipeline validation results based on training using the sitcom database showed that data scraping using search engines does not allow to create a large enough database for training a machine learning model because it creates overfitting. On the other hand, using the MPII training database (which can be collected using data scraping on the YouTube website) allows to obtain better results due to the variety and large number of images. In addition, not distinguishing between the left and right limbs of the human body allows to further improve the results.

The pipeline results between the different training databases are consistent with each other. This allows us to affirm that the system is robust : the evolution of the validation results using one database or the other corresponds to what is expected. If it happens that some highly similar categories are confused with others, grouping the analogous categories subject to confusion helps to avoid them from being confused. In addition, the results show that the most represented body parts such as the torso and the head are more easily detected because they are better represented. These results are in line with the study [16].

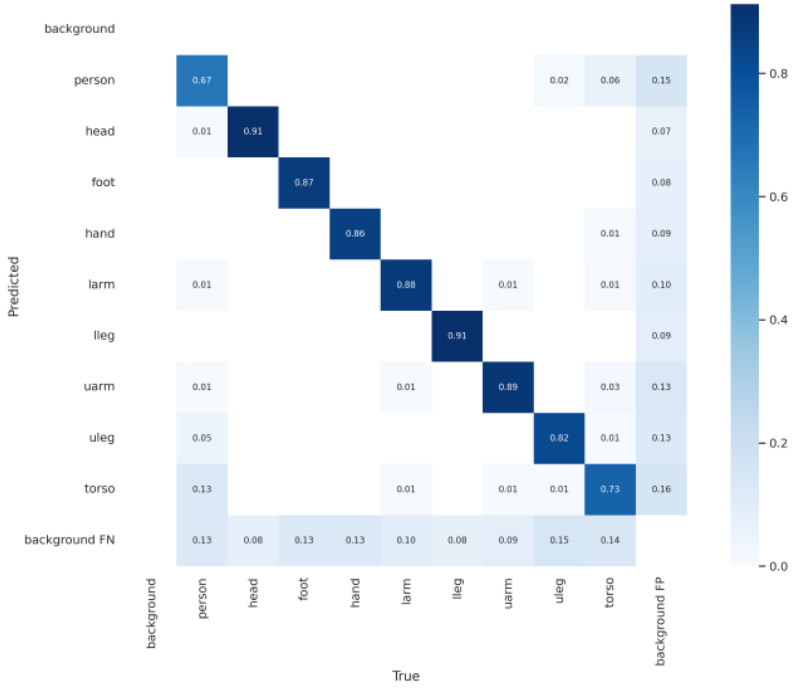


FIGURE 4.14 – Confusion matrix resulting from validating MPII based on MPII training weights without distinguishing between analogous body members

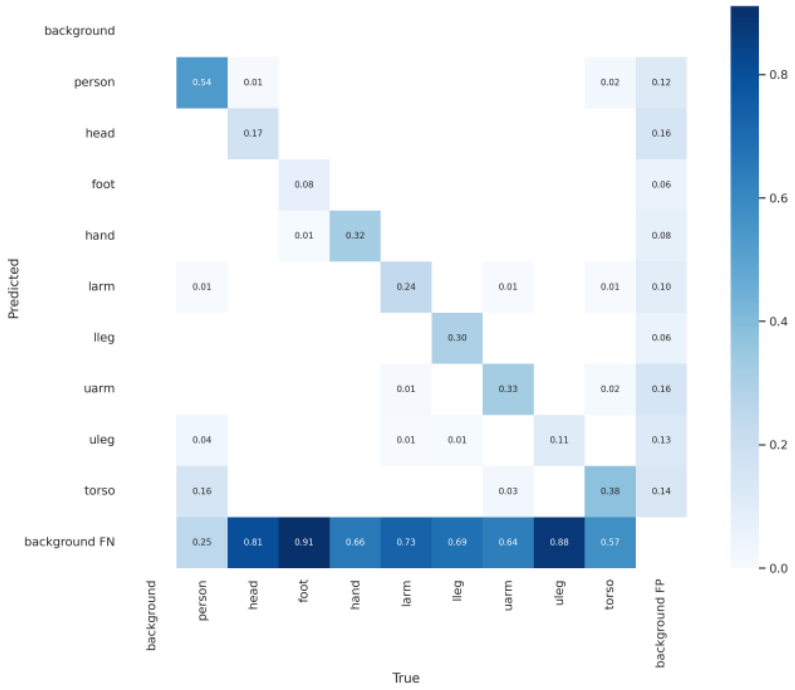


FIGURE 4.15 – Confusion matrix resulting from the validation of Pascal-Part based on MPII training weights without distinguishing between analogous body members

Chapitre 5

Prospects for improvement

5.1 Segmentation

One perspective for improvement would be to use machine learning models that use segmentation masks instead of bounding rectangles parallel to the image sides. Mask R-CNN is a two-stage machine learning model derived from RCNN, Fast and Faster RCNN [56]. As its name suggests, this model uses segmentation masks to detect objects (see figure 5.1). Using segmentation masks instead of bounding rectangles would allow the model to better understand the objects to be detected because they would be more precisely delineated. On the other hand, this technique would also require more precision when detecting human body parts after training. The DensePose dataset from the COCO database would allow to take advantage of Mask-RCNN by adding more data to the Pascal-Part database via segmentation masks.

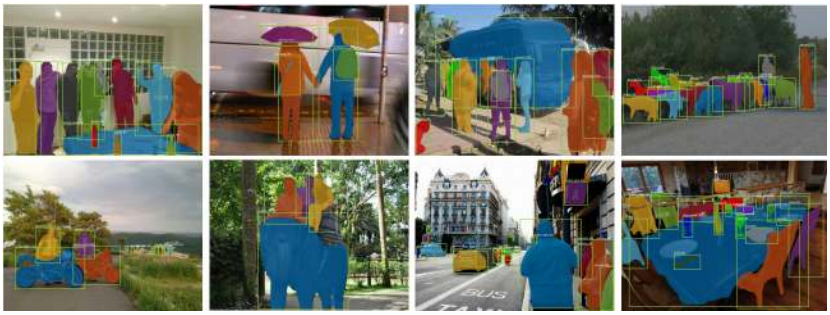


FIGURE 5.1 – Mask R-CNN results on the COCO dataset [56]

A compromise between segmentation masks and bounding rectangles are polygons and quadrilaterals. Researchers have created evolutions of the YOLO version 3 [57] and version 5 [58] algorithms supporting this type of detection. Using polygons increases the accuracy of object localization compared to bounding rectangles while using less storage than segmentation masks in terms of annotation. The difficulty of this method is to transform the segmentation masks provided with the Pascal-Part database into a polygon or quadrilateral. One method to create a quadrilateral from a segmentation mask is to choose the corners of this quadrilateral according to the smallest and largest abscissas and ordinates. However, figure 5.2 shows that this method creates too much data loss. Indeed, the torso is less well represented using the quadrilateral : according to these configurations, learning the body members will be complicated

for the object detection model.

The Ramer–Douglas–Peucker algorithm was invented to reduce the number of points in a polygon [59]. To do this, the algorithm is applied to the perimeter of the segmentation masks only. Experience has shown that a lot of information is lost and the outline is reduced to a one-pixel-thick rectangle before providing a suitable polygon.

a. Segmentation mask b. quadrilateral

FIGURE 5.2 – Creation of a quadrilateral from a segmentation mask by keeping only the extreme coordinates of this mask

5.2 Automatic annotation

The automatic annotation pipeline could be improved in several ways. First, the annotation via MediaPipe skeletons is not perfect : figure **??**.b in section 3.5.4 shows that there is some loss of information regarding the width of the bounding rectangles. Although the small width of the bounding rectangles members in this figure was later improved by setting a minimum threshold, this technique for better delineating the annotated objects remains to be improved. Second, data scraping from the YouTube site could be implemented in the algorithm to increase the size of the training database and limit overfitting. Third, since single-stage object detectors suffer from localization problems for small objects (such as feet or hands), it would be interesting to modify the automatic annotation algorithm to accommodate a two-stage object detector (such as Faster RCNN) to be able to compare the performance of single-stage models and two-stage models. Finally, the use of fine tuning should be considered. Indeed, in a machine learning model, fine tuning allows to only evolve the weights of the model on the last layers of the model and to freeze the weights of the other layers of the model. Freezing most of the first layers avoids retraining them and to use the corresponding weights without modifying them. This method avoids retraining all the weights provided by YOLOv5 (see section 4.5.2) and to use part of the performance of these weights on the created model. Using this technique therefore saves training time and would allow for better performance across the entire model.

Conclusion

In this work, after exploring existing human limb detection databases, a comparison of different image annotation tools for object detection showed that labelme is the best tool. In addition, the object detection machine learning models NanoDet and YOLO were explored in conjunction with the human limb detection system from the MediaPipe framework. The database used to perform the validation tests is the Pascal-Part database which contains 10103 images annotated according to 16 body parts. The results of the comparison between the different human limb detection models show that the 5x version of glsyolo offers the best performance (mAP at 45.1%) while NanoDet requires the least computing resources (mAP at 30.4%). MediaPipe receives a mAP score of 9.11%.

An automatic annotation pipeline was developed based on data scraping, MediaPipe and YOLOv5. Data scraping on the DuckDuckGo search engine using the keywords "sitcom scene" allowed us to collect 800 images containing on average four human beings per image. Annotations of this database (sitcom) using MediaPipe and training of these annotations using YOLO allowed us to create a machine learning model for detecting body parts that is 10.1

Running this automatic annotation pipeline on a larger MPII training dataset from YouTube increases the validation score mAP to 14.1

If the results obtained by the automatic annotation pipeline are not up to our expectations (mAP (IoU at 0.5) at 19.7% for the pipeline against 45.1% for the object detection model YOLOv5 in the best cases), these not being equivalent to the results of comparison of object detection algorithms, various avenues for improvement are proposed. The annotations made using MediaPipe can be improved. A new more precise annotation format can be used such as segmentation masks or polygons. The last layers of the neural network of the object detection model used in the pipeline can be refined (fine tuning).

Bibliographie

[1] Trevor Hastie, Robert Tibshirani et Jerome Friedman. *The Elements of Statistical Learning*. Springer Series in Statistics. New York, NY, USA : Springer New York Inc., 2001.

[2] Keiron O'Shea et Ryan Nash. *An Introduction to Convolutional Neural Networks*. 2015. doi : 10.48550/ARXIV.1511.08458. url : https://arxiv.org/abs/1511.08458.

[3] Athanasios Voulodimos, Nikolaos Doulamis, Anastasios Doulamis, Eftychios Protopapadakis et Diego Andina. « Deep Learning for Computer Vision : A Brief Review ». In : *Intell. Neuroscience* 2018 (jan. 2018). issn : 1687-5265. doi : 10.1155/2018/7068349. url : https://doi.org/10.1155/2018/7068349.

[4] Zhengxia Zou, Zhenwei Shi, Yuhong Guo et Jieping Ye. *Object Detection in 20 Years : A Survey*. 2019. doi : 10.48550/ARXIV.1905.05055. url : https://arxiv.org/abs/1905.05055.

[5] COCOdataset. *COCO - Common Objects in Context*. https://cocodataset.org/. (Consulté le 14/05/2022).

[6] Riza Alp Güler, Natalia Neverova et Iasonas Kokkinos. « DensePose : Dense Human Pose Estimation In The Wild ». In : *CoRR* abs/1802.00434 (2018). arXiv : 1802.00434. url : http://arxiv.org/abs/1802.00434.

[7] Alexander Kirillov, Kaiming He, Ross Girshick, Carsten Rother et Piotr Dollár. *Panoptic Segmentation*. 2018. doi : 10.48550/ARXIV.1801.00868. url : https://arxiv.org/abs/1801.00868.

[8] Justin Johnson. *EECS 498-007 / 598-005 : Deep Learning for Computer Vision | Website for UMich EECS course*. https://web.eecs.umich.edu/~justincj/teaching/eecs498/WI2022/. (Consulté le 23/05/2022). Oct. 2019.

[9] Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard et L. D. Jackel. « Backpropagation Applied to Handwritten Zip Code Recognition ». In : *Neural Computation* 1.4 (jan. 1989), p. 541-551.

[10] P. Viola et M. Jones. « Rapid object detection using a boosted cascade of simple features ». In : *Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition. CVPR 2001*. T. 1. 2001, p. I-I. doi : 10.1109/CVPR.2001.990517.

[11] Ross Girshick, Jeff Donahue, Trevor Darrell et Jitendra Malik. *Rich feature hierarchies for accurate object detection and semantic segmentation*. 2013. doi : 10.48550/ARXIV.1311.2524. url : https://arxiv.org/abs/1311.2524.

[12] Joseph Redmon et Ali Farhadi. « YOLOv1 : Unified, Real-Time Object Detection ». In : *arXiv* (2016).

[13] Shuo Yang, Ping Luo, Chen Change Loy et Xiaoou Tang. *Faceness-Net : Face Detection through Deep Facial Part Responses*. 2017. DOI : 10.48550/ARXIV.1701.08393. URL : https://arxiv.org/abs/1701.08393.

[14] Duc Thanh Nguyen, Wanqing Li et Philip O. Ogunbona. « Human detection from images and videos : A survey ». In : *Pattern Recognition* 51 (2016), p. 148-175. ISSN : 0031-3203. DOI : https://doi.org/10.1016/j.patcog.2015.08.027. URL : https://www.sciencedirect.com/science/article/pii/S0031320315003179.

[15] Wanli Ouyang et Xiaogang Wang. « A discriminative deep model for pedestrian detection with occlusion handling ». In : *2012 IEEE Conference on Computer Vision and Pattern Recognition*. 2012, p. 3258-3265. DOI : 10.1109/CVPR.2012.6248062.

[16] Yi Yang et Deva Ramanan. « Articulated pose estimation with flexible mixtures-of-parts ». In : *CVPR 2011*. 2011, p. 1385-1392. DOI : 10.1109/CVPR.2011.5995741.

[17] Mykhaylo Andriluka, Stefan Roth et Bernt Schiele. « Pictorial structures revisited : People detection and articulated pose estimation ». In : *2009 IEEE Conference on Computer Vision and Pattern Recognition*. 2009, p. 1014-1021. DOI : 10.1109/CVPR.2009.5206754.

[18] Sohaib Laraba, Matei Mancas et Bernard Gosselin. *Visual Processing and Smart Spaces*. Information, Signal, et Intelligence Artificielle Lab, Faculté polytechnique de Mons. 2021. Chap. Motion Capture : human activity recognition and interaction.

[19] Ce Zheng, Wenhan Wu, Chen Chen, Taojiannan Yang, Sijie Zhu, Ju Shen, Nasser Kehtarnavaz et Mubarak Shah. *Deep Learning-Based Human Pose Estimation : A Survey*. 2020. DOI : 10.48550/ARXIV.2012.13392. URL : https://arxiv.org/abs/2012.13392.

[20] Jonathan Tompson, Arjun Jain, Yann LeCun et Christoph Bregler. *Joint Training of a Convolutional Network and a Graphical Model for Human Pose Estimation*. 2014. DOI : 10.48550/ARXIV.1406.2984. URL : https://arxiv.org/abs/1406.2984.

[21] Valentin Bazarevsky, Ivan Grishchenko, Karthik Raveendran, Tyler Zhu, Fan Zhang et Matthias Grundmann. *BlazePose : On-device Real-time Body Pose tracking*. 2020. DOI : 10.48550/ARXIV.2006.10204. URL : https://arxiv.org/abs/2006.10204.

[22] Zhe Cao, Tomas Simon, Shih-En Wei et Yaser Sheikh. *Realtime Multi-Person 2D Pose Estimation using Part Affinity Fields*. 2016. DOI : 10.48550/ARXIV.1611.08050. URL : https://arxiv.org/abs/1611.08050.

[23] Leonid Pishchulin, Eldar Insafutdinov, Siyu Tang, Bjoern Andres, Mykhaylo Andriluka, Peter Gehler et Bernt Schiele. *DeepCut : Joint Subset Partition and Labeling for Multi Person Pose Estimation*. 2015. DOI : 10.48550/ARXIV.1511.06645. URL : https://arxiv.org/abs/1511.06645.

[24] Hao-Shu Fang, Guansong Lu, Xiaolin Fang, Jianwen Xie, Yu-Wing Tai et Cewu Lu. *Weakly and Semi Supervised Human Body Part Parsing via Pose-Guided Knowledge Transfer*. 2018. DOI : 10.48550/ARXIV.1805.04310. URL : https://arxiv.org/abs/1805.04310.

[25] Wenguan WANG, Zhijie ZHANG, Siyuan QI, Jianbing SHEN, Yanwei PANG et Ling SHAO. « Learning Compositional Neural Information Fusion for Human Parsing ». In : *CoRR* abs/2001.06804 (2020). arXiv : 2001.06804. URL : https://arxiv.org/abs/2001.06804.

[26] Mykhaylo ANDRILUKA, Leonid PISHCHULIN, Peter GEHLER et Bernt SCHIELE. « 2D Human Pose Estimation : New Benchmark and State of the Art Analysis ». In : *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. Juin 2014.

[27] Kevin LIN, Lijuan WANG, Kun LUO, Yinpeng CHEN, Zicheng LIU et Ming-Ting SUN. « Cross-Domain Complementary Learning Using Pose for Multi-Person Part Segmentation ». In : *IEEE Transactions on Circuits and Systems for Video Technology* 31.3 (mars 2021), p. 1066-1078. DOI : 10.1109/tcsvt.2020.2995122. URL : https://doi.org/10.1109%2Ftcsvt.2020.2995122.

[28] Peike LI, Yunqiu XU, Yunchao WEI et Yi YANG. *Self-Correction for Human Parsing*. 2019. DOI : 10.48550/ARXIV.1910.09777. URL : https://arxiv.org/abs/1910.09777.

[29] Xiaodan LIANG, Ke GONG, Xiaohui SHEN et Liang LIN. *Look into Person : Joint Body Parsing & Pose Estimation Network and A New Benchmark*. 2018. DOI : 10.48550/ARXIV.1804.01984. URL : https://arxiv.org/abs/1804.01984.

[30] Wenguan WANG, Zhijie ZHANG, Siyuan QI, Jianbing SHEN, Yanwei PANG et Ling SHAO. « Learning Compositional Neural Information Fusion for Human Parsing ». In : (2020). DOI : 10.48550/ARXIV.2001.06804. URL : https://arxiv.org/abs/2001.06804.

[31] Gabriel L. OLIVEIRA, Wolfram BURGARD et Thomas BROX. « Efficient deep models for monocular road segmentation ». In : *2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2016, p. 4885-4891. DOI : 10.1109/IROS.2016.7759717.

[32] Xiaojie LI, Lu YANG, Qing SONG et Fuqiang ZHOU. « Detector-in-Detector : Multi-Level Analysis for Human-Parts ». In : *CoRR* abs/1902.07017 (2019). arXiv : 1902.07017. URL : http://arxiv.org/abs/1902.07017.

[33] M. EVERINGHAM, L. VAN GOOL, C. K. I. WILLIAMS, J. WINN et A. ZISSERMAN. « The Pascal Visual Object Classes (VOC) Challenge ». In : *International Journal of Computer Vision* 88.2 (juin 2010), p. 303-338.

[34] Xianjie CHEN, Roozbeh MOTTAGHI, Xiaobai LIU, Sanja FIDLER, Raquel URTASUN et Alan L. YUILLE. « Detect What You Can : Detecting and Representing Objects using Holistic Models and Body Parts ». In : *CoRR* abs/1406.2031 (2014). arXiv : 1406.2031. URL : http://arxiv.org/abs/1406.2031.

[35] Tsung-Yi LIN, Michael MAIRE, Serge BELONGIE, Lubomir BOURDEV, Ross GIRSHICK, James HAYS, Pietro PERONA, Deva RAMANAN, C. Lawrence ZITNICK et Piotr DOLLÁR. *Microsoft COCO : Common Objects in Context*. 2014. DOI : 10.48550/ARXIV.1405.0312. URL : https://arxiv.org/abs/1405.0312.

[36] Glenn JOCHER et al. *ultralytics/yolov5 : v6.1 - TensorRT, TensorFlow Edge TPU and OpenVINO Export and Inference*. Version v6.1. Fév. 2022. DOI : 10.5281/zenodo.6222936. URL : https://doi.org/10.5281/zenodo.6222936.

[37] Bo Zhao. « Web Scraping ». In : mai 2017, p. 1-3. ISBN : 978-3-319-32001-4. DOI : 10.1007/978-3-319-32001-4_483-1.

[38] *À propos de DuckDuckGo*. https://duckduckgo.com/about. (Consulté le 09/05/2022).

[39] *Sources | DuckDuckGo Help Pages*. https://help.duckduckgo.com/duckduckgo-help-pages/results/sources/. (Consulté le 19/05/2022).

[40] Rafael Padilla, Sergio L. Netto et Eduardo A. B. da Silva. « A Survey on Performance Metrics for Object-Detection Algorithms ». In : *2020 International Conference on Systems, Signals and Image Processing (IWSSIP)*. 2020, p. 237-242. DOI : 10.1109/IWSSIP48289.2020.9145130.

[41] Haidi Zhu, Haoran Wei, Baoqing Li, Xiaobing Yuan et Nasser Kehtarnavaz. « A Review of Video Object Detection : Datasets, Metrics and Methods ». In : *Applied Sciences* 10.21 (2020). ISSN : 2076-3417. DOI : 10.3390/app10217834. URL : https://www.mdpi.com/2076-3417/10/21/7834.

[42] Tom Fawcett. « An introduction to ROC analysis ». In : *Pattern Recognition Letters* 27.8 (2006). ROC Analysis in Pattern Recognition, p. 861-874. ISSN : 0167-8655. DOI : https://doi.org/10.1016/j.patrec.2005.10.010. URL : https://www.sciencedirect.com/science/article/pii/S016786550500303X.

[43] Johnathan Hui. *mAP (mean Average Precision) for Object Detection | by Jonathan Hui | Medium*. https://jonathan-hui.medium.com/map-mean-average-precision-for-object-detection-45c121a31173. (Consulté le 13/05/2022). Mars 2018.

[44] Phil Ferriere. *philferriere/cocoapi : Clone of COCO API - Dataset @ http ://cocodataset.org/ - with changes to support Windows build and python3*. https://github.com/philferriere/cocoapi. (Consulté le 14/05/2022). Oct. 2018.

[45] Joseph Redmon et Ali Farhadi. « YOLO9000 : Better, Faster, Stronger ». In : *arXiv preprint arXiv :1612.08242* (2016).

[46] Joseph Redmon et Ali Farhadi. « YOLOv3 : An Incremental Improvement ». In : *arXiv* (2018).

[47] RangiLyu. *NanoDet-Plus : Super fast and high accuracy lightweight anchor-free object detection model*. https://github.com/RangiLyu/nanodet. 2021.

[48] Zhi Tian, Chunhua Shen, Hao Chen et Tong He. *FCOS : Fully Convolutional One-Stage Object Detection*. 2019. DOI : 10.48550/ARXIV.1904.01355. URL : https://arxiv.org/abs/1904.01355.

[49] Vijay Singh. *What is Frameworks ? [Definition] Types of Frameworks*. https://hackr.io/blog/what-is-frameworks. (Consulté le 09/05/2022). Mars 2022.

[50] Camillo Lugaresi et al. *MediaPipe : A Framework for Building Perception Pipelines*. 2019. DOI : 10.48550/ARXIV.1906.08172. URL : https://arxiv.org/abs/1906.08172.

[51] Google. *Home - mediapipe*. https://google.github.io/mediapipe/. (Consulté le 09/05/2022).

[52] Shawn Tng. *Multi-Person Pose Estimation with Mediapipe | by Shawn Tng | Medium*. https://shawntng.medium.com/multi-person-pose-estimation-with-mediapipe-52e6a60839dd. (Consulté le 10/05/2022). Mars 2021.

[53] *Définitions : flops - Dictionnaire de français Larousse*. https://www.larousse.fr/dictionnaires/francais/flops/34200. (Consulté le 17/05/2022).

[54] *Sitcom : Définition simple et facile du dictionnaire*. https://www.linternaute.fr/dictionnaire/fr/definition/sitcom/. (Consulté le 10/05/2022).

[55] Yan GOBEIL. *Making an image dataset from Youtube videos | by Yan Gobeil | Towards Data Science*. https://towardsdatascience.com/making-an-image-dataset-from-youtube-videos-5116252d20a3. (Consulté le 04/06/2022). Août 2019.

[56] Kaiming HE, Georgia GKIOXARI, Piotr DOLLÁR et Ross GIRSHICK. *Mask R-CNN*. 2017. DOI : 10.48550/ARXIV.1703.06870. URL : https://arxiv.org/abs/1703.06870.

[57] MING71. *GitHub - XinzeLee/PolygonObjectDetection : This repository is based on Ultralytics/yolov5, with adjustments to enable polygon prediction boxes.* https://github.com/XinzeLee/PolygonObjectDetection. (Consulté le 21/05/2022). Août 2020.

[58] XINZELEE. *GitHub - XinzeLee/PolygonObjectDetection : This repository is based on Ultralytics/yolov5, with adjustments to enable polygon prediction boxes.* https://github.com/XinzeLee/PolygonObjectDetection. (Consulté le 21/05/2022). Avr. 2022.

[59] Dilip K. PRASAD, Maylor K.H. LEUNG, Chai QUEK et Siu-Yeung CHO. « A novel framework for making dominant point detection methods non-parametric ». In : *Image and Vision Computing* 30.11 (2012), p. 843-859. ISSN : 0262-8856. DOI : https://doi.org/10.1016/j.imavis.2012.06.010. URL : https://www.sciencedirect.com/science/article/pii/S0262885612000984.